

Detecting Cyber Threats at Scale:

Anomaly Detection in Network and System Logs Using Machine Learning

Catherine Kaye Dela Chica
CS 406, Personal Project with Ami Gates
Final Submission: June 5, 2026

Table of Contents

Table of Contents.....	1
Introduction.....	3
Machine Learning Topics.....	5
Links to Artifacts.....	5
Thinking Questions.....	5
The Datasets.....	6
Data Set 1: CIC-IDS2017.....	6
Data set 2: UNSW-NB15.....	7
How These Datasets Will Be Used.....	8
Data Cleaning, Preparation, and EDA.....	9
Data Set 1: CIC-IDS2017.....	9
Data Set 2: UNSW-NB15.....	10
KMeans Clustering and PCAKmeans Clustering: Data Set 1.....	11
k =3.....	12
k =5.....	12
k =7.....	12
Best k for Data set 1.....	13
PCA (Hierarchical Agglomerative Clustering):.....	14
Regression Models:.....	15
Data set 1: Linear and Logistic Regression preparation.....	15
Linear Regression Results.....	16
Logistic Regression Results.....	16
Data set 2: Linear and Logistic Regression preparation.....	17
Linear Regression Results.....	18
Logistic Regression Results.....	19
Discussion of Regression Modeling Results:.....	19
Classification Models Part 1:.....	20
Decision Tree Analysis: What is it?.....	20
Data Set 1: Preparation.....	20
Results.....	21
Random Forest.....	24
Data Set 2: Preparation.....	26
Results.....	27
Random Forest.....	28
Discussion of Results.....	29
Classification Models Part 2:.....	29
What's Naive Bayes?.....	29
Date Set 1: Naive Bayes Preparation.....	30

Gaussian NB.....	30
Multinomial NB.....	31
Naive Bayes Results.....	32
Date Set 2:Naive Bayes Preparation.....	33
Gaussian NB.....	34
Multinomial NB.....	35
Naive Bayes Results.....	36
Analyses.....	37
Phase 1.....	37
Phase 2:.....	39
Results.....	41
Performance Comparison Across All Models.....	41
Random Forest Results.....	42
Decision Tree Results.....	42
Logistic Regression Results.....	43
Linear Regression Results.....	44
K-Means Clustering and Hierarchical Clustering Results.....	45
Naive Bayes Results.....	46
Overall Results.....	47
Thinking Questions	47
Q1.....	47
Q2.....	47
Q3.....	48
Q4.....	48
Q5.....	49
Q6.....	49
Q7.....	49
Q8.....	50
Q9.....	50
Q10.....	51
Conclusion.....	52
References.....	54

Introduction



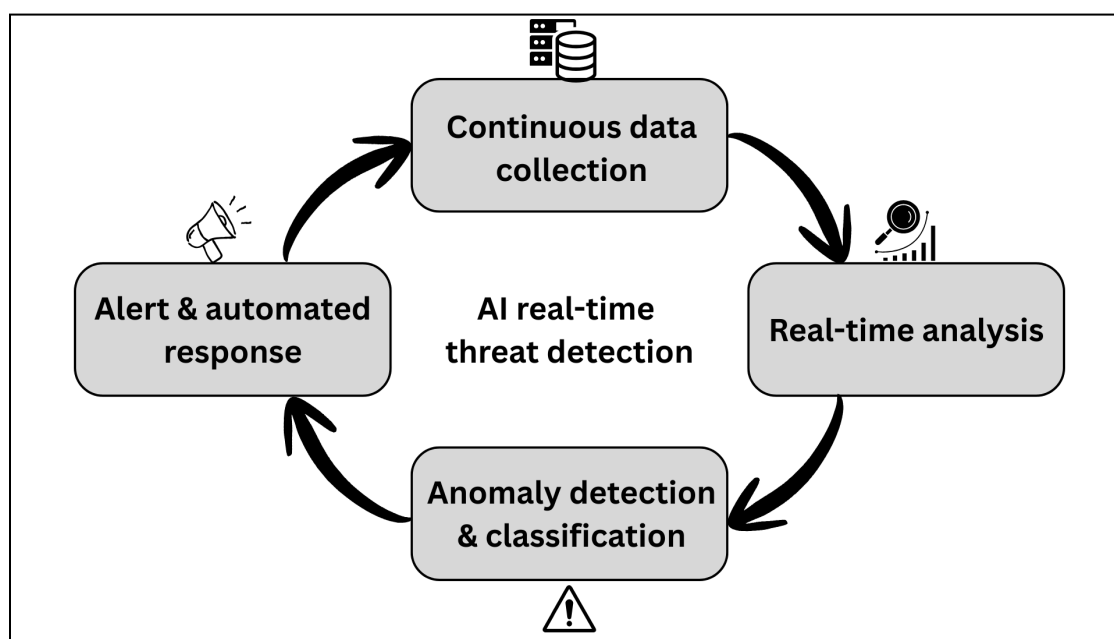
Types of Security Logs | A Complete Guide for Cybersecurity Monitoring in 2026

Cybersecurity has become one of the most significant challenges in today's increasingly digital world. Nearly every aspect of modern society now depends on technology, including banking, healthcare, education, communication, transportation, and online services. As more personal and organizational information is stored digitally, cyber attacks continue to grow in both frequency and complexity. Organizations are no longer only defending against obvious attacks, but also against hidden threats designed to blend into normal system behavior. This creates a major challenge for security teams, as malicious activity can often remain unnoticed among millions of routine actions occurring every day across networks, applications, and devices.

One of the primary tools used to monitor digital systems is security logging. Every interaction within a system, such as user logins, file access, application activity, or network communication, generates records known as security logs. These logs serve as a detailed history of system activity and are essential for identifying suspicious behavior, unauthorized access attempts, and potential system breaches. However, modern systems produce an overwhelming amount of log data continuously. The volume and complexity of this information make it extremely difficult for human analysts to manually monitor and interpret all activity in real time. As a result, organizations increasingly require automated solutions capable of processing and analyzing large scale data efficiently.

This project focuses on the use of artificial intelligence and machine learning to improve real time cybersecurity monitoring and threat detection. Rather than relying entirely on

traditional rule based systems, machine learning enables systems to identify unusual behavior patterns automatically by learning from large amounts of continuously collected data. The process begins with continuous data collection from systems and network activity. That information is then analyzed in real time, allowing machine learning models to identify anomalies and classify potentially suspicious behavior. Once threats or unusual activities are detected, automated alerts and responses can be triggered to help organizations respond more quickly and effectively to potential security incidents.



Threat Detection Lifecycle

The importance of this approach continues to grow as cyber attacks become more advanced and unpredictable. Traditional cybersecurity systems are often limited because they primarily detect known attack signatures or predefined threats. However, many modern cyber attacks evolve rapidly and may not follow previously recognized patterns. Machine learning provides a more adaptive and intelligent approach by identifying behaviors that appear abnormal even if they have never been encountered before. This allows organizations to move beyond reactive cybersecurity practices and toward more proactive threat detection strategies capable of identifying risks earlier in the attack lifecycle.

Beyond the technical goals of the project, this work addresses a broader societal issue related to trust, privacy, and digital security. Individuals and organizations depend heavily on secure systems to protect sensitive information and maintain reliable services. When cybersecurity measures fail, the consequences can include financial loss, data breaches, operational disruption, and loss of public trust. By improving the ability to monitor systems continuously and detect threats in real time, this project contributes toward creating safer and more secure digital environments. Ultimately, it demonstrates how intelligent technologies can support cybersecurity professionals in protecting users, organizations, and critical systems in an increasingly connected world.

Machine Learning Topics

A full breakdown of machine learning topics can be located on the [CS406 Project Repo Wiki](#).

The objectives of this project are to apply the full data science and machine learning workflow to a real world dataset while demonstrating competency in data preparation, unsupervised learning, supervised modeling, and comparative model evaluation. Specifically, the project will:

- Perform Data Gathering
- Conduct Data Cleaning and Preparation
- Apply Unsupervised Learning Techniques (Clustering and PCA)
- Implement Linear Regression Modeling
- Implement Logistic Regression Modeling
- Apply Decision Tree and Tree-Ensemble Classification Methods
- Implement Bayesian Classification Techniques
- Apply Support Vector Machine Classification
- Implement Ensemble Classification Methods
- Compare and Interpret All Modeling Approaches

Links to Artifacts

- [Public Github Repo](#)
- [Original Proposal](#)
- [Machine Learning Topics](#)
- [Term Timeline](#)
- The Datasets
 - [CIC-IDS2017](#)
 - [UNSW-NB15](#)

Thinking Questions

The following questions are designed to guide this project beyond simply building and evaluating machine learning models, but instead focus on understanding their real world impact and usefulness. While technical performance is important, the true value of this system lies in how effectively it can detect threats, support decision making, and operate within real cybersecurity environments. These questions aim to explore not only how well the models perform, but also what their results mean in practical terms, such as how they handle risk, how they would be used by analysts, and how they could be integrated into existing systems. By answering these questions, the project evaluates both the technical success and the real world

applicability of machine learning for cybersecurity anomaly detection

1. Can this system detect cyber threats early enough to prevent real damage?:
 - a. This question evaluates whether the model can identify suspicious activity quickly enough to allow organizations to respond before a breach or attack causes harm.
2. What is the acceptable balance between false positives and false negatives?
 - a. This explores the trade off between flagging too many normal events as threats versus missing actual attacks, which directly impacts system usability and risk.
3. Which types of anomalies or attacks are easiest and hardest to detect?
 - a. This helps identify where the model performs well and where it struggles, revealing limitations in detecting subtle or complex threats.
4. Can unsupervised learning detect previously unseen or evolving cyber threats?
 - a. This examines whether clustering and PCA can uncover new attack patterns that were not included in the training data.
5. Which machine learning model provides the best performance for anomaly detection?
 - a. This compares different models to determine which approach is most effective based on accuracy, precision, recall, and overall reliability.
6. How well do the models generalize to new, unseen data?
 - a. This evaluates whether the trained models can maintain performance when applied to different datasets or real world environments.
7. How does dimensionality reduction (PCA) impact anomaly detection performance?
 - a. This investigates whether reducing feature complexity improves model efficiency and the ability to detect meaningful patterns.
8. How can this system be integrated into a real world cybersecurity workflow?
 - a. This considers how the model's outputs could be used within tools like SIEM systems to support real time monitoring and decision making.
9. How would cybersecurity analysts interpret and act on the model's results?
 - a. This focuses on the usability of the system and whether its outputs provide clear, actionable insights for human users.
10. What impact could this system have on improving cybersecurity in today's digital world?
 - a. This explores the broader significance of the project, including its potential to protect sensitive data, reduce risk, and strengthen trust in digital systems.

The Datasets

Data Set 1: CIC-IDS2017

This CIC-IDS2017 dataset is a widely used benchmark dataset for intrusion detection research, designed to address limitations in older cybersecurity datasets. It contains both normal and malicious network traffic in a realistic environment between July 3–7, 2017. The dataset includes raw packet captures (PCAPs) and processed labeled network flow data (CSV files) with over 80 extracted features such as IPs, ports, protocols, timestamps, and attack labels. Unlike older datasets, it emphasizes realistic user behavior, generated using a profiling

system that simulates activities like web browsing, email, file transfer protocol (FTP), and secure shell (SSH) across multiple operating systems and devices.

A key strength of CIC-IDS2017 is its diverse and labeled attack scenarios, including brute force (FTP/SSH), denial of service/distributed denial of service (DoS/DDoS) attacks, web attacks (cross site scripting (XSS), SQL injection), Heartbleed, botnets, infiltration, and port scanning, executed across different days and time windows. This structured labeling makes it ideal for machine learning workflows from training to evaluation, especially for anomaly detection and classification tasks. Additionally, the dataset includes full network topology, realistic traffic volumes, and detailed metadata, making it one of the most comprehensive datasets for building and evaluating modern intrusion detection systems.

The CICIDS2017 is an ideal fit for this project because it provides a large, realistic collection of labeled network traffic that closely reflects modern cybersecurity environments, including both normal activity and a wide range of attack types such as DDoS, brute force, and infiltration. Its preprocessed MachineLearningCSV format makes it accessible for demonstrating the full machine learning workflow from data cleaning and feature engineering to applying both supervised models (like logistic regression, SVM, and decision trees) and unsupervised techniques (like PCA and clustering for anomaly detection). This balance between realism and usability allows the project to effectively explore detecting rare and hidden threats within high-volume log data, while also supporting model comparison, evaluation, and real world applicability, directly aligning with the goal of identifying anomalies in network and system logs at scale.

Data set 2: UNSW-NB15

The UNSW-NB15 dataset is a modern network intrusion detection dataset created in a controlled cyber range environment using the IXIA PerfectStorm tool. It combines realistic normal network activity with synthetic attack traffic, captured as about 100 GB of raw packet data (PCAP files). From this traffic, researchers extracted 49 network flow features using tools like Argus and Bro-IDS, producing labeled CSV datasets that distinguish between normal behavior and nine different attack types, including DoS, exploits, backdoors, reconnaissance, and worms.

The dataset contains over 2.5 million records and is structured to support machine learning workflows, with predefined training and testing splits for model development and evaluation. It also includes supporting files such as ground truth labels and event lists, making it suitable for tasks like anomaly detection, classification, and network forensics. Overall, UNSW-NB15 is designed to reflect modern network threats more accurately than older datasets, providing a comprehensive and realistic benchmark for cybersecurity research.

The UNSW-NB15 is a strong fit for this project because it offers a modern, well-balanced dataset that captures realistic network traffic alongside a diverse set of contemporary attack types, making it ideal for evaluating how well machine learning models detect cyber threats beyond a single data source. Unlike older datasets, it reflects more current attack behaviors and includes rich feature representations, allowing you to effectively train and test both supervised

and unsupervised models. Most importantly, it is valuable for assessing model generalization, helping you demonstrate whether your anomaly detection system can maintain performance on new, unseen data, an essential component of building reliable, real world cybersecurity solutions.

How These Datasets Will Be Used

The CICIDS2017 will be used as the primary dataset by selecting and combining multiple files (Tuesday, Wednesday, and Friday DDoS) to create a diverse dataset that includes both normal network activity and a variety of cyber attacks. This dataset supports the full machine learning workflow, including data cleaning (handling missing and infinite values), feature selection, and preprocessing. Both supervised models (such as logistic regression, decision trees, and support vector machines) and unsupervised techniques (such as PCA and clustering) can then be applied to detect anomalies and classify malicious behavior. Model performance can be evaluated using metrics such as accuracy, precision, and recall, while also analyzing which types of attacks are easier or more difficult to detect.

To further assess model robustness, the UNSW-NB15 will be used as a separate validation dataset to test generalization on new, unseen data. Because UNSW-NB15 contains different feature distributions and attack patterns, it provides a more realistic evaluation of how well the models perform outside the original training environment. Comparing results across both datasets enables analysis of model reliability and real-world applicability, particularly in detecting cyber threats at scale and adapting to evolving attack behaviors.

Data Cleaning, Preparation, and EDA

Data Set 1: CIC-IDS2017

<pre>**** MISSING VALUES **** Flow_Bytes/s 201 Flow_Duration 0 Destination_Port 0 Total_Backward_Packets 0 Total_Length_of_Fwd_Packets 0 Total_Length_of_Bwd_Packets 0 Total_Fwd_Packets 0 Fwd_Packet_Length_Max 0 Fwd_Packet_Length_Min 0 Fwd_Packet_Length_Std 0 Fwd_Packet_Length_Mean 0 Bwd_Packet_Length_Max 0 Bwd_Packet_Length_Min 0 Bwd_Packet_Length_Mean 0 Bwd_Packet_Length_Std 0 Flow_Packets/s 0 Flow_IAT_Mean 0 Flow_IAT_Std 0 Flow_IAT_Max 0 Flow_IAT_Min 0 dtype: int64 **** DUPLICATE ROWS **** 24065 Now Cleaning data... Removed 264 rows with missing or infinite values. Removed 24019 duplicate rows. Shape after cleaning: (421626, 79) **** LABEL DISTRIBUTION **** Label 0 412476 1 9150 Name: count, dtype: int64</pre>	<pre>**** MISSING VALUES **** Flow_Bytes/s 1008 Flow_Duration 0 Destination_Port 0 Total_Backward_Packets 0 Total_Length_of_Fwd_Packets 0 Total_Length_of_Bwd_Packets 0 Total_Fwd_Packets 0 Fwd_Packet_Length_Max 0 Fwd_Packet_Length_Min 0 Fwd_Packet_Length_Std 0 Fwd_Packet_Length_Mean 0 Bwd_Packet_Length_Max 0 Bwd_Packet_Length_Min 0 Bwd_Packet_Length_Mean 0 Bwd_Packet_Length_Std 0 Flow_Packets/s 0 Flow_IAT_Mean 0 Flow_IAT_Std 0 Flow_IAT_Max 0 Flow_IAT_Min 0 dtype: int64 **** DUPLICATE ROWS **** 81909 Now Cleaning data... Removed 1297 rows with missing or infinite values. Removed 80914 duplicate rows. Shape after cleaning: (610492, 79) **** LABEL DISTRIBUTION **** Label 0 416736 1 193756 Name: count, dtype: int64</pre>	<pre>**** MISSING VALUES **** Flow_Bytes/s 28 Flow_Duration 0 Destination_Port 0 Total_Backward_Packets 0 Total_Length_of_Fwd_Packets 0 Total_Length_of_Bwd_Packets 0 Total_Fwd_Packets 0 Fwd_Packet_Length_Max 0 Fwd_Packet_Length_Min 0 Fwd_Packet_Length_Std 0 Fwd_Packet_Length_Mean 0 Bwd_Packet_Length_Max 0 Bwd_Packet_Length_Min 0 Bwd_Packet_Length_Mean 0 Bwd_Packet_Length_Std 0 Flow_Packets/s 0 Flow_IAT_Mean 0 Flow_IAT_Std 0 Flow_IAT_Max 0 Flow_IAT_Min 0 dtype: int64 **** DUPLICATE ROWS **** 6888 Now Cleaning data... Removed 122 rows with missing or infinite values. Removed 6867 duplicate rows. Shape after cleaning: (184044, 79) **** LABEL DISTRIBUTION **** Label 0 182096 1 1948 Name: count, dtype: int64</pre>
Tuesday, Before Cleaning	Wednesday, Before Cleaning	Friday, Before Cleaning

This project requires working with large-scale network traffic data from the CICIDS2017 dataset to detect cyber threats using machine learning. Because these logs are extremely high volume and contain noise, redundancy, and inconsistencies, the data must be carefully cleaned and prepared before modeling. Based on the implemented code, the cleaning process includes standardizing column names and labels, replacing infinite values, removing rows with missing values, and eliminating duplicate records. Feature engineering is also applied by removing highly correlated features (correlation > 0.90) and low variance features to reduce redundancy and improve efficiency. The goal is to retain columns that capture meaningful network behavior, such as packet lengths, flow duration, and timing metrics, because these features are essential for identifying patterns that distinguish normal traffic from malicious activity. Keeping these informative and consistent features ensures the dataset remains realistic and useful for answering key questions about detection accuracy, generalization, and anomaly detection.

After cleaning, all datasets (Tuesday, Wednesday, and Friday) showed significant improvements in quality and structure. For the Tuesday dataset, the original 445,909 rows were reduced by removing 264 rows with missing or infinite values and 24,019 duplicate rows, resulting in 421,626 clean records, along with about 30 highly correlated features removed. The Wednesday dataset underwent the largest transformation: started with 692,703 rows, removed 1,297 invalid rows and 80,914 duplicates, left 610,492 rows, and eliminated 32 correlated features. Similarly, the Friday dataset decreased from 191,033 rows to 184,044 after removing 122 invalid rows and 6,867 duplicates, with 32 correlated features removed. These steps significantly reduced noise and redundancy across all datasets.

Overall, the “before” datasets contained missing values, duplicate entries, and highly redundant features that could negatively impact model performance. After cleaning and feature engineering, the datasets became more compact, consistent, and meaningful, with the feature space reduced from 79 columns to the mid-40s after correlation filtering and further to the mid-30s after low-variance filtering. This transformation improves computational efficiency, reduces overfitting, and enhances the model’s ability to accurately detect anomalies. As a result, the cleaned datasets are better suited for machine learning, enabling stronger performance, improved generalization to unseen data, and more reliable real-world cybersecurity applications.

Data Set 2: UNSW-NB15

The UNSWNB15 dataset already provides separate training and testing sets, which eliminates the need for a manual train/test split as required with the CICIDS2017 dataset. Because these datasets are pre-partitioned and commonly used for benchmarking, they are considered well-structured and largely precleaned for machine learning tasks. This allows the focus of the project to shift toward feature engineering and model evaluation rather than extensive data cleaning. In this approach, the training set is used to fit and train machine learning models such as logistic regression, decision trees, and support vector machines, while the testing set is reserved strictly for evaluating performance on unseen data. This setup directly supports the project’s goal of assessing how well models generalize to new environments, as the testing set simulates real-world network traffic that the model has not previously encountered.

```
Training set loaded: (82332, 45)
Testing set loaded: (175341, 45)

X_train shape: (82332, 43)
X_test shape: (175341, 43)
y_train shape: (82332,)
y_test shape: (175341,)

Numeric columns: 40
Categorical columns: ['proto', 'service', 'state']

After scaling and encoding:
X_train_processed shape: (82332, 191)
X_test_processed shape: (175341, 191)

After low-variance filtering:
X_train_final shape: (82332, 54)
X_test_final shape: (175341, 54)

**** FINAL SUMMARY ****
Original training rows: 82332
Cleaned training shape: (82332, 45)
Prepared training shape: (82332, 2)

Original testing rows: 175341
Cleaned testing shape: (175341, 45)
Prepared testing shape: (175341, 2)
```

UNSWNB15 Training/Test Cleaning

Although the dataset is already prepared, light preprocessing and feature engineering are still applied to ensure consistency across the modeling pipeline. This includes standardizing numerical features using techniques such as StandardScaler and encoding categorical variables (proto, service, and state) through one-hot encoding. Importantly, all preprocessing steps are fit only on the training data and then applied to the testing data to prevent data leakage and maintain a fair evaluation. The dataset initially contained 82,332 training rows and 175,341 testing rows, each with 45 columns. After separating the target variable (label), the feature sets consisted of 43 input features, including 40 numeric and 3 categorical variables. Following scaling and encoding, the feature space expanded to 191 features due to the transformation of categorical variables into binary columns.

To improve efficiency and reduce redundancy, low-variance feature selection was applied, reducing the feature set from 191 to 54 meaningful features. This step removed features that provided little variation across the dataset and therefore contributed minimally to model performance. As a result, the final prepared datasets consist of 82,332 training instances and 175,341 testing instances, each with 54 engineered features plus the target label. Overall, the “before” datasets were already clean but dimensional, while the “after” datasets are more compact, informative, and optimized for machine learning. This transformation enhances computational efficiency, reduces noise, and improves the model’s ability to detect anomalies and generalize effectively to real-world cybersecurity scenarios.

KMeans Clustering and PCA

Kmeans Clustering: Data Set 1

In this portion of the code, KMeans clustering was applied to the combined CICIDS2017 dataset (Tuesday, Wednesday, and Friday) to identify natural groupings within the network traffic data. After ensuring consistency across datasets by selecting only common columns, the data was cleaned, converted to numeric form, and standardized using StandardScaler to prevent features with larger scales from dominating the clustering process. KMeans was then implemented using multiple values of k (3, 5, and 7) to explore different clustering structures. For each value of k , the model assigned cluster labels to the data, and the distribution of points across clusters was analyzed. Silhouette scores were calculated to evaluate how well-separated and cohesive the clusters were, helping determine the most appropriate number of clusters.

```
*****
KMEANS RESULTS WITHOUT PCA: k=3
*****
Cluster counts:
0    238421
1    684413
2    391328
Name: count, dtype: int64
Silhouette Score: 0.2741

*****
KMEANS RESULTS WITHOUT PCA: k=5
*****
Cluster counts:
0    277924
1    471367
2    294423
3    53671
4    118777
Name: count, dtype: int64
Silhouette Score: 0.3744

*****
KMEANS RESULTS WITHOUT PCA: k=7
*****
Cluster counts:
0    43458
1    472221
2    248070
3    60700
4    21429
5    265359
6    104933
Name: count, dtype: int64
Silhouette Score: 0.3933

KMEANS Pandas Clustering Without PCA
```

To enhance interpretability, Principal Component Analysis (PCA) was incorporated to reduce the dataset to three dimensions. This allowed for both 2D and 3D visualizations of the clusters, making it easier to observe how data points group together and whether distinct

patterns emerge. The best performing k-value was selected based on the highest silhouette score, and the resulting clusters were visualized. Additionally, anomaly detection was performed by calculating the distance of each data point to its nearest cluster centroid; points with the largest distances (top 5%) were flagged as potential anomalies. This step helped identify unusual network behaviors that may correspond to cyber threats.

k=3

After applying PCA, the clustering results improve significantly, with the silhouette score increasing to 0.5407. This indicates much stronger separation between clusters compared to the non PCA results. The clusters are also more balanced in size, suggesting that PCA has successfully reduced noise and emphasized the most important patterns in the data. With $k = 3$, the model is now able to clearly distinguish broader categories of network traffic, likely separating major behavioral groups such as normal traffic and different types of attacks. However, while separation is strong, the lower number of clusters still limits the model's ability to capture finer-grained patterns.

```
*****
KMEANS RESULTS AFTER PCA: k=3
*****
Cluster counts:
0      459394
1      467405
2      289363
Name: count, dtype: int64
Silhouette Score: 0.5407
```

k=5

```
*****
KMEANS RESULTS AFTER PCA: k=5
*****
Cluster counts:
0      276912
1      471628
2      278958
3      166363
4       22301
Name: count, dtype: int64
Silhouette Score: 0.6530
```

At $k = 5$, the silhouette score further improves to 0.6530, indicating well-defined and clearly separated clusters. The distribution of data points shows both large and moderately sized clusters, as well as a smaller cluster that may represent more unusual or rare activity. This suggests that PCA has made the underlying structure of the data more apparent, allowing KMeans to better partition the dataset into meaningful groups. The clusters at this level likely represent distinct categories of network behavior, offering a strong balance between

generalization and detail, and making this configuration highly effective for exploratory analysis.

k=7

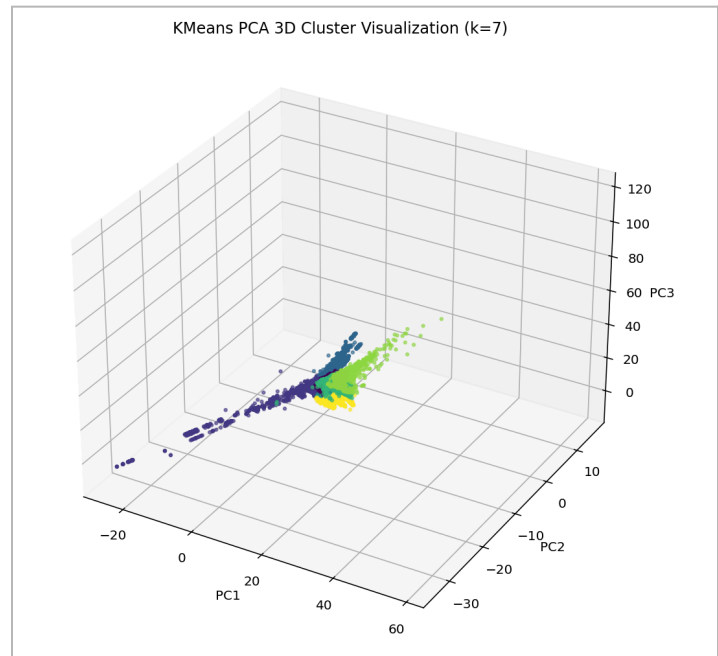
With $k = 7$, the clustering achieves the highest silhouette score of 0.3933 before PCA, indicating the best separation among the three tested values in the original feature space. The clusters become more granular, with several smaller groups emerging alongside larger ones. This suggests that the dataset contains multiple distinct patterns, including possibly rare or specialized traffic

```
*****
KMEANS RESULTS AFTER PCA: k=7
*****
Cluster counts:
0      265206
1      464835
2      276228
3      112308
4       35445
5        1944
6       60196
Name: count, dtype: int64
Silhouette Score: 0.6563
```

behaviors. The presence of smaller clusters is especially important, as they may represent anomalies or less frequent attack types. While the improvement is noticeable, the silhouette score still indicates only moderate separation, highlighting the limitations of clustering in high-dimensional space without dimensionality reduction.

Best k for Data set 1

With $k = 7$, the model achieves the highest silhouette score of 0.6563, making it the best-performing configuration. The clusters are both well-separated and more granular, capturing subtle variations in the data. Notably, the presence of very small clusters (such as one with only 1,944 data points) suggests that the model is identifying rare or anomalous patterns, which is particularly valuable in a cybersecurity context. The improvement over $k = 5$ is small but meaningful, indicating that the additional clusters help capture finer distinctions without significantly reducing cluster quality. Therefore, $k = 7$ is chosen as the best value because it provides the optimal balance of high separation, detailed grouping, and the ability to detect potential anomalies within the network traffic data.



The 3D PCA KMeans visualization with $k = 7$ directly supports the core goal of this project, detecting cyber threats hidden within massive volumes of system and network log data. In this visualization, each cluster represents a distinct pattern of network behavior, and the clear separation between clusters shows that the model is successfully identifying meaningful structure without relying on labeled data. The larger, denser clusters likely represent normal system activity, such as routine user behavior and expected traffic patterns, while the smaller or more isolated clusters reflect less frequent patterns that may correspond to anomalies or potential attack signatures. This demonstrates that the model can effectively distinguish between normal and unusual behavior, which is essential for threat detection in environments where malicious activity is often buried within large amounts of legitimate data.

More importantly, these results highlight the strength of unsupervised learning for proactive cybersecurity. Instead of relying on predefined rules or known attack signatures, the model identifies deviations from normal behavior, making it capable of detecting new or evolving threats that traditional systems might miss. The ability to isolate smaller, less frequent clusters is particularly valuable, as these may represent suspicious events such as abnormal traffic spikes or unauthorized access attempts. By reducing the data using PCA, the model improves cluster separation and makes these anomalies easier to identify. In a real-world

setting, this approach allows analysts to prioritize investigations by focusing on outlier groups, leading to faster response times and reducing the likelihood of undetected breaches. Overall, these results demonstrate that the system is both technically effective and practically valuable for improving anomaly detection and strengthening cybersecurity defenses.

PCA (Hierarchical Agglomerative Clustering):

```
*****
HIERARCHICAL AGGLOMERATIVE CLUSTERING
*****

Using rows for Agglomerative:
50000

Agglomerative counts (k=3)
0    11662
1    18631
2    19707
Name: count, dtype: int64

Agglomerative counts (k=5)
0    11662
1    11747
2    18631
3     7403
4     557
Name: count, dtype: int64

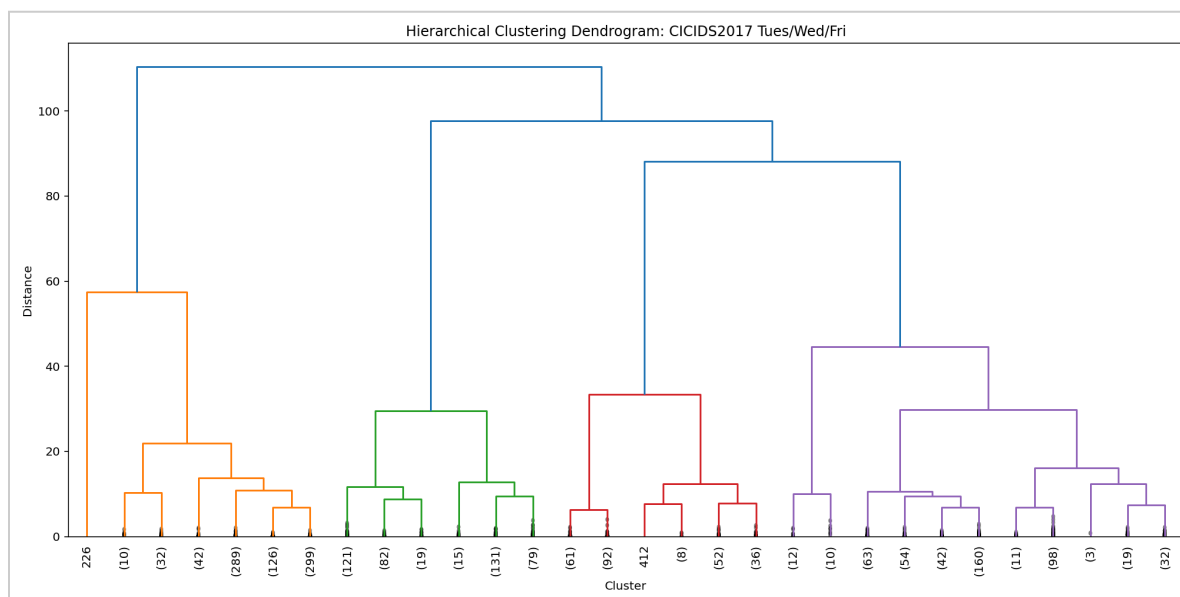
Agglomerative counts (k=7)
0     2
1    2137
2    5266
3     555
4    11662
5    11747
6    18631
Name: count, dtype: int64

Using dendrogram sample size:
2000
```

In the second part of the code, Hierarchical Agglomerative Clustering was applied to further analyze the structure of the data and compare it with the KMeans results. Because the dataset is large, a random subset of the PCA-transformed data was selected to ensure computational efficiency. A connectivity matrix was created using `kneighbors_graph` to define relationships between nearby data points, which improves clustering performance by considering local structure. The Agglomerative Clustering algorithm was then applied with different values of k (3, 5, and 7) using Ward linkage, which minimizes variance within clusters as they are merged.

To visualize the hierarchical relationships between data points, a dendrogram was generated using the `scipy` linkage function. This dendrogram illustrates how clusters are formed step-by-step, showing the distances at which clusters merge and providing insight into the natural grouping of the data. By examining both the cluster counts and the

dendrogram structure, it becomes possible to understand how the data is organized at different levels of granularity. This approach complements KMeans by offering a hierarchical perspective, helping validate clustering patterns and revealing how smaller clusters combine into larger ones, which is especially useful for understanding complex network behaviors.



The hierarchical clustering dendrogram illustrates how network traffic data is grouped based on similarity, showing the step by step merging of data points into clusters as distance increases. The vertical height of each merge represents the dissimilarity between clusters, where shorter branches indicate closely related data points and taller branches indicate greater differences. In this result, many clusters form at lower distances, suggesting strong similarity within groups, while several clusters only merge at higher distances, indicating the presence of distinct and less similar patterns in the dataset. This structure highlights that the data is not uniform and contains multiple levels of grouping, ranging from highly similar behavior to more separated and unique patterns.

In the context of cyber threat detection, these findings are significant because the clusters that merge at higher distances may represent unusual or anomalous behavior that differs from typical system activity. This supports the goal of identifying hidden threats within large volumes of log data, where malicious actions are often subtle and embedded within normal operations. Unlike KMeans, which requires a predefined number of clusters, hierarchical clustering reveals the natural structure of the data and allows analysis at multiple levels of granularity. This makes it especially useful for understanding how different behaviors relate to one another and for identifying where abnormal activity begins to diverge. Overall, the dendrogram demonstrates that meaningful patterns and potential anomalies can be uncovered through unsupervised learning, contributing to more effective and proactive cybersecurity analysis.

Regression Models:

Data set 1: Linear and Logistic Regression preparation

```
Cleaned Dataset Shape: (1216162, 56)
```

```
Binary Label Counts:
```

```
Binary_Label
```

```
0    1011308
```

```
1     204854
```

```
Name: count, dtype: int64
```

```
Features Used:
```

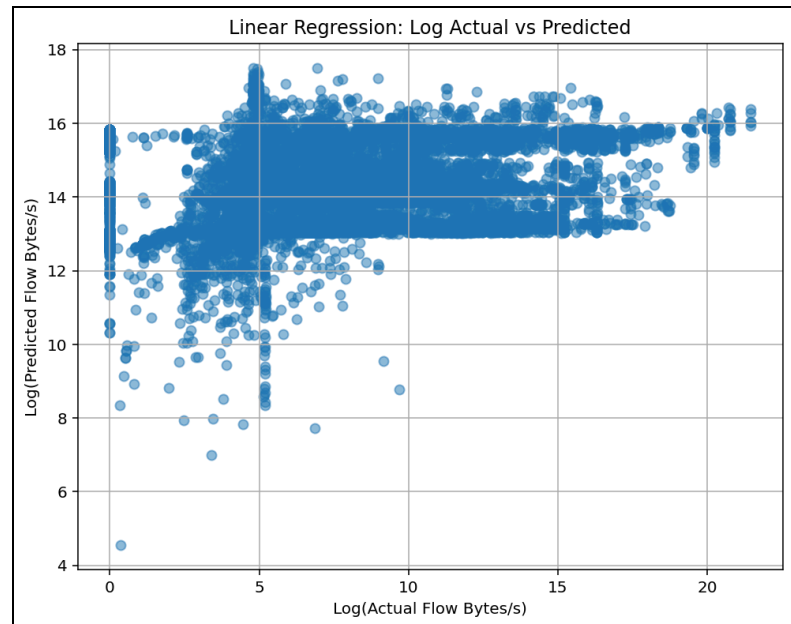
- Destination_Port
- Flow_Duration
- Total_Fwd_Packets
- Total_Length_of_Fwd_Packets
- Fwd_Packet_Length_Max
- Fwd_Packet_Length_Min
- Fwd_Packet_Length_Mean
- Bwd_Packet_Length_Max
- Bwd_Packet_Length_Min
- Flow_Bytes/s

To prepare the CICIDS2018 dataset for linear and logistic regression, the data set was standardized by the column names by removing extra spaces. It was then cleaned by replacing infinite values with missing values, keeping only numeric columns, and filling missing numeric values with each column's median. This was important because both Linear Regression and Logistic Regression require numerical input and cannot properly train on missing or infinite values. After cleaning, we created a Binary_Label column from the existing numeric Label column, where 0 represents benign traffic and 1 represents malicious traffic. We also selected CICIDS2017 network traffic features such as flow duration, packet counts, byte totals,

packet length means, and flow rates to use as model inputs.

Linear Regression Results

The Linear Regression model was used to predict Flow_Bytes/s using several network traffic features from the CICIDS2017 dataset, including destination port, flow duration, packet counts, and packet length measurements. After training the model, it produced an R^2 value of 0.0107, which means the model only explained about 1% of the variation in network traffic volume. The model also had fairly large error values, with an MAE of about 3.05 million and an RMSE of about 34 million. These results suggest that network



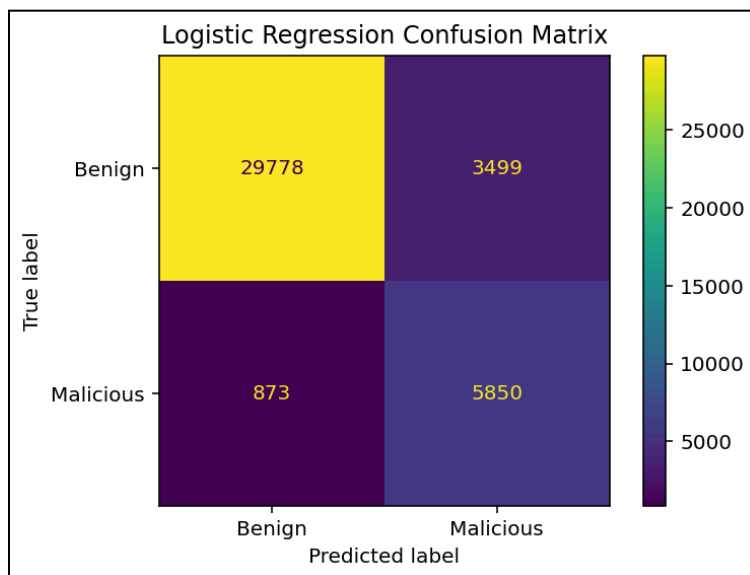
traffic behavior is much more complex than a simple linear relationship can capture. While the model was able to generate predictions, the differences between the predicted and actual values were often quite large. This indicates that the selected features alone are not enough for Linear Regression to accurately model network traffic patterns, and that more advanced machine learning techniques may be needed to better understand and detect unusual activity within the dataset.

The log-transformed Actual vs. Predicted graph supports what the metrics showed. Instead of the points following a clear diagonal pattern, they are spread out across the graph and form several horizontal bands. This means the model often predicted values within a similar range, even when the actual traffic values were very different. In other words, the model struggled to capture the wide variation that exists in real network traffic. Based on both the graph and the performance metrics, Linear Regression was not very effective for predicting network traffic behavior in this dataset. However, it still served as a useful starting point by establishing a baseline for normal traffic patterns and helping demonstrate why more advanced machine learning models are often needed for cybersecurity anomaly detection.

Logistic Regression Results

The Logistic Regression model was used to classify network flows as either benign or malicious. Overall, the model achieved an accuracy of 89.07%, which shows that it performed well on the testing data. Looking deeper into the classification report, the model achieved a precision of 97% and a recall of 89% for benign traffic. For malicious traffic, it achieved a precision of 63% and a recall of 87%. The high recall for malicious traffic is especially important because it means the model was able to identify most of the attack-related network flows instead

of missing them. While there were some false positives, the model generally did a good job distinguishing between normal and suspicious traffic.



The confusion matrix helps visualize how well the model performed. Out of 33,277 benign flows, the model correctly classified 29,778 as benign and incorrectly labeled 3,499 as malicious. For malicious traffic, the model correctly identified 5,850 attacks and missed 873 malicious flows. Compared to the Linear Regression model, Logistic Regression was much more effective for this project because it directly classifies traffic as either normal or malicious, which aligns with the goal of intrusion and anomaly detection. Overall, the results show that Logistic Regression provides a strong baseline for detecting suspicious network activity and can be a useful tool for helping cybersecurity analysts identify potential threats within the CICIDS2017 dataset.

Data set 2: Linear and Logistic Regression preparation

The UNSW-NB15 training and testing datasets were combined into a single dataset to create a larger and more complete collection of network traffic records for analysis. Data preparation included replacing infinite values, filling missing numeric values with the median of each feature, and selecting numerical attributes related to network activity, such as packet counts, byte counts, traffic loads, and connection duration. A binary classification label was used, where 0 represented normal traffic and 1 represented attack traffic. Because the dataset contains a large number of records, a sample size of 200,000 observations was used with a fixed random seed to reduce processing time while ensuring consistent and

```
Labels Found:
label
1    164673
0     93000
Name: count, dtype: int64

Cleaned Dataset Shape: (257673, 41)

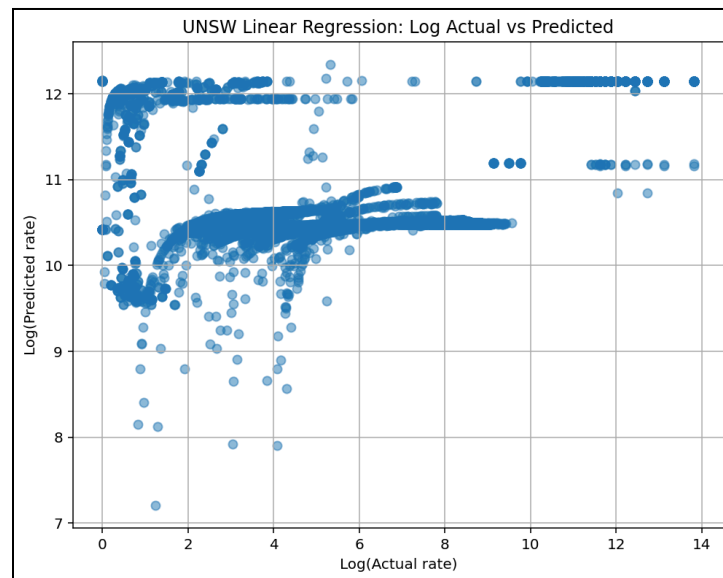
Binary Label Counts:
Binary_Label
1    164673
0     93000
Name: count, dtype: int64

Features Used:
- dur
- sbytes
- dbytes
- sttl
- dttl
- sload
- dload
- spkts
- dpkts
- rate
```

reproducible results. The analysis focused on two objectives: determining whether network traffic features could be used to predict connection rate through linear regression and evaluating whether those same features could accurately distinguish between normal and malicious traffic using logistic regression. These models provide insight into the relationship between network behavior and cyberattack detection.

Linear Regression Results

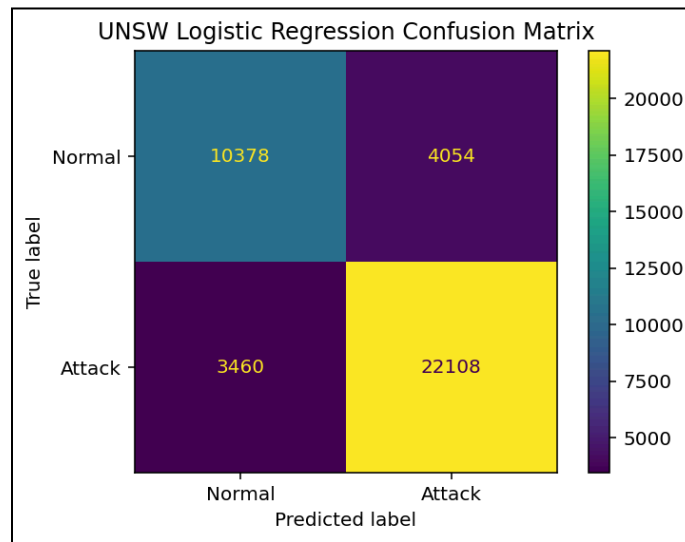
The linear regression model was used to predict the network connection rate (rate) using several network traffic features, including duration, packet counts, byte counts, and TTL values. The model produced an R^2 value of 0.3397, meaning that about 34% of the variation in connection rate can be explained by the features included in the model. While this shows that there is some relationship between the variables, the relatively high MAE (76,071.99) and RMSE (131,794.79) indicate that the predictions were often far from the actual values. This suggests that the selected features do not fully explain changes in connection rate and that other factors may also influence the target variable.



The scatterplot of the log-transformed actual and predicted values helps visualize these results. If the model were highly accurate, the points would closely follow a diagonal pattern. Instead, the graph shows several horizontal bands and a noticeable spread of points, indicating that many records received similar predicted values even when their actual rates were different. This pattern suggests that the relationship between the network features and connection rate is more complex than a simple linear relationship. Based on both the evaluation metrics and the visualization, linear regression was only moderately successful at predicting connection rates within the UNSW-NB15 dataset.

Logistic Regression Results

The logistic regression model performed much better when classifying traffic as either Normal or Attack. The model achieved an overall accuracy of 81.22%, showing that it was able to correctly classify most network traffic records. The classification report also shows strong performance across both classes, with higher scores for attack traffic than normal traffic. This indicates that the model was generally more effective at identifying malicious activity, which is important in a cybersecurity setting where detecting attacks is often the primary goal.



The confusion matrix provides a clearer picture of the model's performance. A total of 22,108 attack records were correctly identified, while 3,460 attacks were missed and classified as normal traffic. For normal traffic, 10,378 records were correctly classified, while 4,054 were incorrectly flagged as attacks. Although some classification errors occurred, the number of correct predictions was much higher than the number of incorrect predictions. Looking at both the confusion matrix and the classification metrics, logistic regression proved to be a useful model for detecting malicious network activity in the UNSW-NB15 dataset. Overall, the classification results were much stronger than the linear regression results, suggesting that the network traffic features are better suited for identifying attacks than for predicting an exact connection rate.

Discussion of Regression Modeling Results:

The results from both datasets showed a clear difference between the two machine learning approaches. Linear regression was able to find some relationships between the network traffic features and the target variables, but it struggled to make accurate predictions. This was especially noticeable in the visualizations, where the predicted values were spread out and often failed to closely match the actual values. Although the UNSW-NB15 dataset produced a better R^2 value than the CICIDS2017 dataset, neither model was able to capture the full complexity of network traffic behavior. These results suggest that network activity is influenced by many

factors that do not follow a simple linear pattern, making it difficult for linear regression to serve as a reliable method for detecting anomalies on its own.

Logistic regression produced much stronger results across both datasets and aligned more closely with the overall goal of anomaly detection. Rather than trying to predict an exact traffic value, the model focused on determining whether network activity was normal or malicious. In both datasets, the majority of attack traffic was correctly identified, showing that features such as packet counts, byte totals, traffic rates, and connection characteristics contain useful information for distinguishing between legitimate and suspicious behavior. While some attacks were still missed and some normal traffic was incorrectly flagged, the overall performance demonstrates that machine learning can effectively support anomaly detection efforts. One of the biggest takeaways from this project is that classification models are generally more useful than regression models when the goal is to identify potential threats. Instead of predicting how traffic should behave, logistic regression was able to recognize patterns associated with attacks and separate them from normal network activity, making it a much stronger choice for cybersecurity applications.

Classification Models Part 1:

Decision Tree Analysis: What is it?

Decision tree analysis is a machine learning method that classifies data by creating a series of rule based decisions similar to a flowchart. The model analyzes features within a dataset and determines which conditions best separate different outcomes or categories. In cybersecurity and intrusion detection, decision trees are commonly used to identify whether network traffic is normal or malicious by evaluating factors such as packet size, flow duration, packet rates, and traffic behavior. Because decision trees are easy to interpret and visualize, they are especially useful for understanding how predictions and classifications are made.

This method is highly relevant to the project because the CICIDS2017 dataset is being used to analyze and classify network traffic. A decision tree can help detect cyberattacks such as DDoS attacks, brute force attempts, and port scans by learning patterns from the dataset's network flow features. The model can automatically determine which traffic characteristics are most important for identifying suspicious activity, making it an effective tool for intrusion detection and cybersecurity analysis. Additionally, the visual structure of a decision tree makes it easier to explain the model's results and understand how attack classifications are determined.

Data Set 1: Preparation

The Decision Tree Classification model will be used to analyze the CICIDS2017 network intrusion dataset in order to determine how effectively a supervised machine learning model can classify network traffic as benign or malicious. By using traffic-flow features such as packet sizes, flow duration, packet rates, byte rates, and TCP flag information, the model will learn

patterns associated with normal traffic and attack activity. The project will compare at least two decision tree models with different parameters to evaluate accuracy, confusion matrices, false positives, false negatives, and overall reliability on unseen test data. Using the the `cicds2017_tues_wed_fri_cleaned.csv` file from earlier, These results will help show which traffic patterns are easier or harder for the model to classify and whether decision trees could support cybersecurity analysts by flagging suspicious network activity for further investigation.

```
Dataset After Cleaning:
(1213510, 56)
  Destination_Port  Flow_Duration  ...  Bwd_Packet_Length_Mean
Max_Packet_Length
0                88             640  ...              0.0
0.0
1                88             900  ...              0.0
0.0
2                88            1205  ...              0.0
0.0
3                88             511  ...              0.0
0.0
4                88             773  ...              0.0
0.0
```

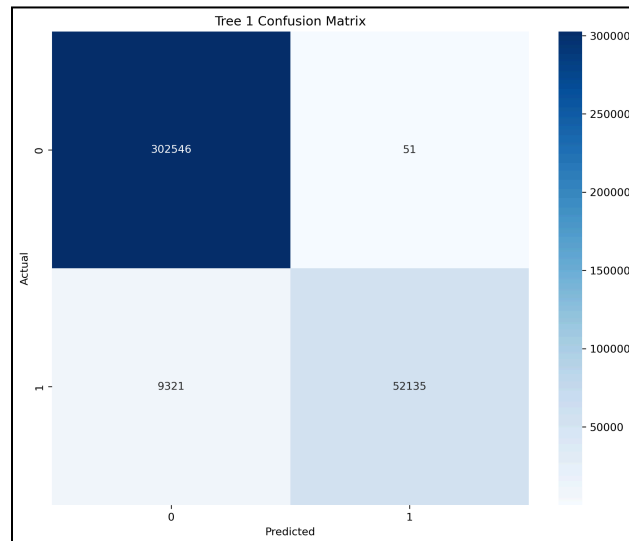
Despite being cleaned already, the code was run through additional steps. This included extra spaces were removed from column names, infinity values were converted to missing values, blank cells were replaced with NaN, duplicate rows were removed, and all feature columns were converted to numeric data types. Columns that were completely empty were removed, and the `Dataset_Day` column was excluded from the machine learning model because it was descriptive rather than predictive. Missing numeric values were filled with zeros to ensure the dataset could be processed correctly by Scikit-learn's `DecisionTreeClassifier`. After cleaning, the dataset was separated into features (X) and labels (y) for supervised learning.

After preprocessing and cleaning, the CICIDS2017 dataset contained 1,213,510 rows of network traffic data with 55 numeric feature columns used for machine learning classification. The dataset labels consisted of two categories, 0 and 1, representing the binary classification of benign and malicious traffic. To evaluate model performance on unseen data, the dataset was divided into separate training and testing sets using Scikit-learn's `train_test_split` function. The training dataset contained 849,457 rows and was used to train the decision tree models, while the testing dataset contained 364,053 rows and was reserved for evaluating model accuracy and prediction performance. Separating the data into training and testing sets helps ensure that the models are evaluated fairly and can generalize to new network traffic rather than simply memorizing the training data.

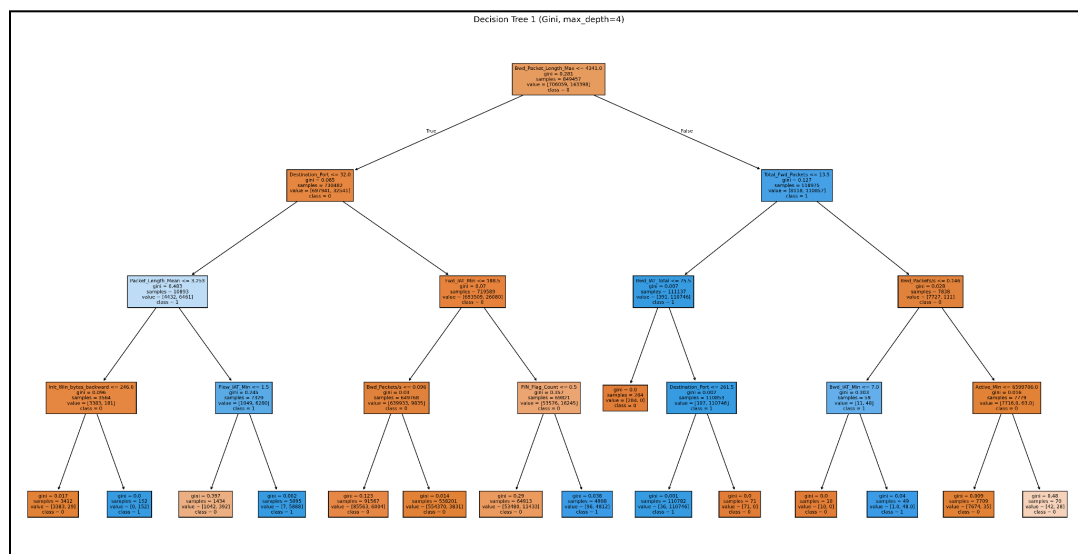
Results

Decision Tree 1's confusion matrix shows how the first decision tree model performed using the Gini impurity criterion with a maximum depth of 4. The model correctly classified 302,546 benign traffic records and 52,135 malicious traffic records. Only 51 benign records were incorrectly identified as malicious traffic, showing that the model handled normal network traffic very accurately. However, 9,321 malicious traffic records were incorrectly classified as benign, meaning some attacks were missed by the model. In cybersecurity, false negatives are

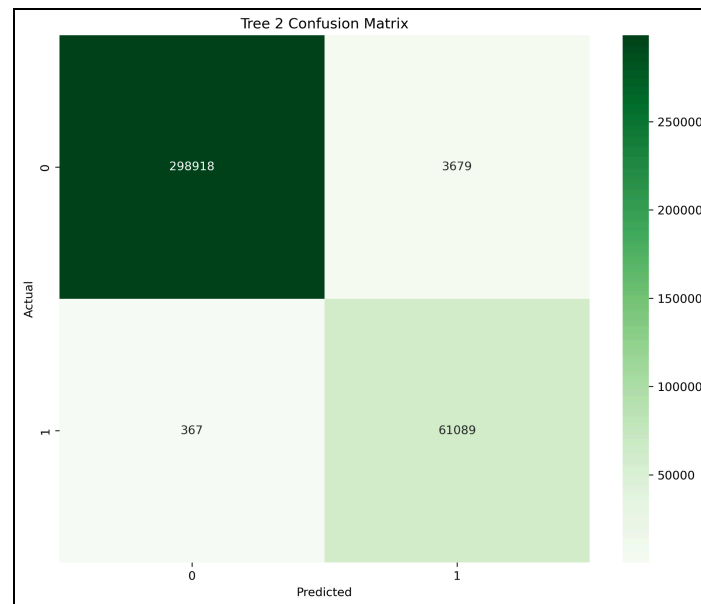
important because missed attacks could allow harmful activity to continue without detection. Overall, the confusion matrix shows that Tree 1 performed well at identifying benign traffic while still detecting most malicious traffic. The darker sections of the matrix visually show where the majority of predictions were classified correctly.



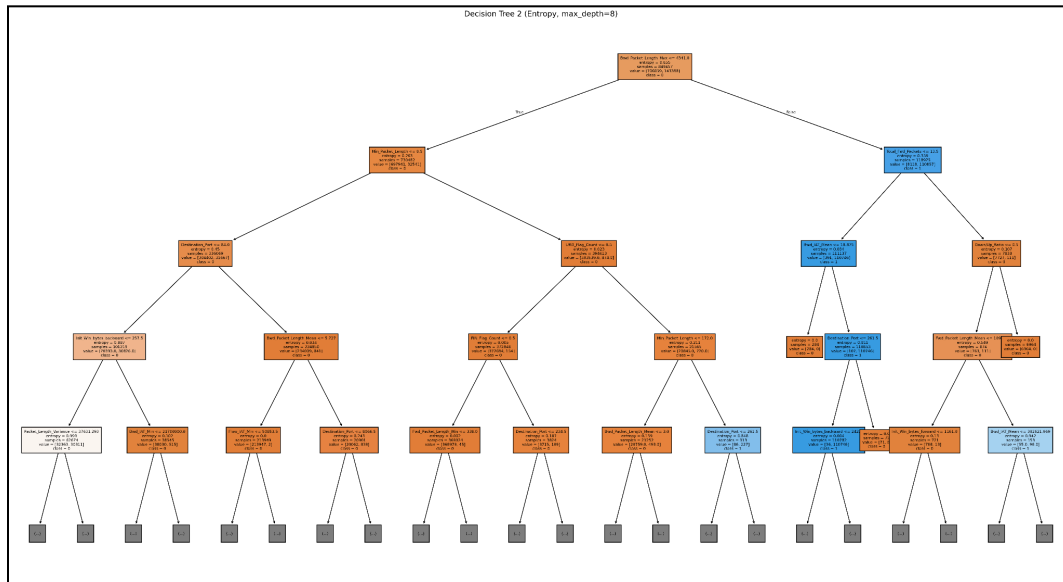
Decision Tree 1's visualization shows how the first decision tree model classified network traffic by following different feature-based decision rules. The root node begins with the feature `Bwd_Packet_Length_Max`, which means it was one of the most important features used to separate benign and malicious traffic. Each node displays the split condition, Gini impurity value, number of samples, class distribution, and predicted class. Orange nodes mostly represent benign traffic (class 0), while blue nodes represent malicious traffic (class 1). Since the tree depth was limited to 4, the structure stayed smaller and easier to read. Features such as `Destination_Port`, `Packet_Length_Mean`, `Fwd_IAT_Min`, and `Bwd_Packets/s` were used multiple times to separate traffic behavior. The visualization helps show how the model followed different branching paths to classify network traffic using measurable flow features.



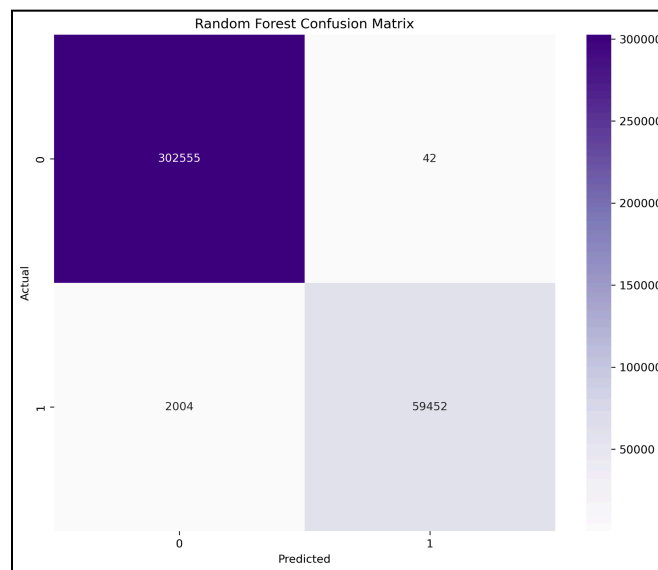
Decision Tree 2's confusion matrix shows the improved performance of the second decision tree model, which used the entropy criterion with a deeper tree structure (max_depth=8). The model correctly classified 298,918 benign traffic records and 61,089 malicious traffic records. Compared to Tree 1, the number of false negatives dropped significantly from 9,321 to only 367, meaning far fewer attacks were missed. The number of false positives increased slightly to 3,679, but the large reduction in missed attacks greatly improved the model's overall threat detection ability. This resulted in an overall accuracy of 98.89%, which was higher than the first model. The confusion matrix visually shows a stronger balance between attack detection and classification accuracy, making Tree 2 more effective for identifying suspicious network traffic.



Decision Tree 2's visualization shows a more detailed and complex decision tree structure compared to Tree 1. This model used the entropy criterion with a deeper maximum depth, allowing more branches and decision paths to capture more complex relationships within the CICIDS2017 network traffic dataset. Similar to Tree 1, the root node starts with Bwd_Packet_Length_Max, showing that this feature remains highly important for classification. The deeper structure also allowed additional features such as Min_Packet_Length, URG_Flag_Count, Bwd_IAT_Mean, Down/Up_Ratio, and Destination_Port to help separate traffic patterns more effectively. The orange and blue node colors again represent benign and malicious classifications, while the entropy values show how pure each node became after splitting. Compared to the simpler Tree 1 structure, Tree 2 contains more levels and more decision boundaries, which helps explain why the second model achieved higher detection accuracy and missed fewer malicious traffic records.



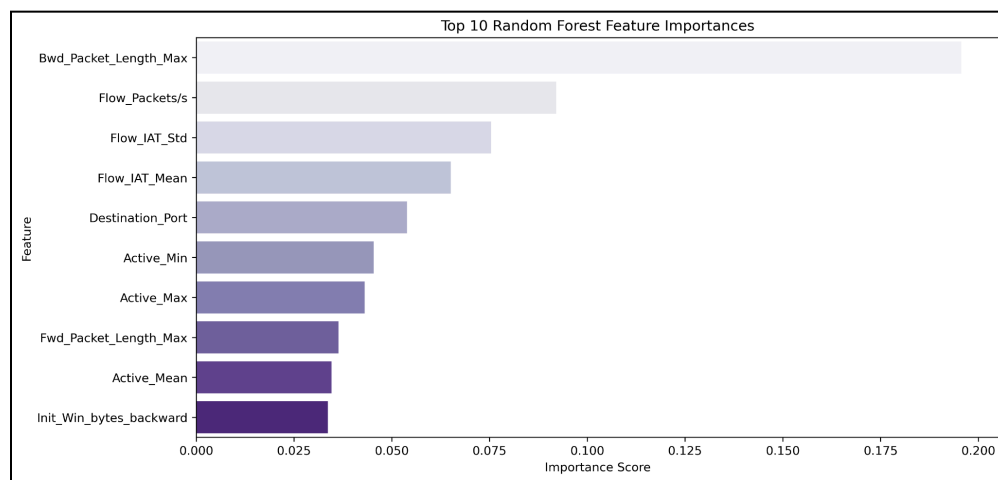
Random Forest



The Random Forest model produced the best overall performance out of all the models tested on the CICIDS2017 dataset, achieving an accuracy of 99.44%. The classification report shows that the model performed extremely well when identifying both benign and malicious network traffic. Benign traffic achieved almost perfect precision and recall scores, while malicious traffic achieved a precision score of 100% and a recall score of 97%. This means the model was very effective at correctly identifying cyberattacks while also keeping false positives and false negatives low. Compared to the individual decision tree models, the Random Forest

model showed stronger overall reliability because it combined predictions from multiple decision trees instead of relying on only one tree structure. This helped reduce overfitting and allowed the model to recognize more detailed traffic patterns within the dataset. Overall, the Random Forest model produced the strongest balance between accuracy, attack detection, and overall classification performance for the CICIDS2017 intrusion detection dataset.

Feature importance analysis was included to help show which network traffic features had the biggest impact on the Random Forest model's predictions. Since Random Forest models use many decision trees together, the feature importance scores help explain which variables the model relied on most when separating benign traffic from malicious activity, making the model easier to interpret and connect back to real cybersecurity behavior. The feature importance graph showed that `Bwd_Packet_Length_Max` was the most important feature, followed by



features such as `Flow_Packets/s`, `Flow_IAT_Std`, `Flow_IAT_Mean`, and `Destination_Port`. These features are related to packet behavior, timing patterns, and network communication flow, showing that the model relied heavily on traffic-flow statistics to identify malicious activity. Features related to connection activity, such as `Active_Min`, `Active_Max`, and `Active_Mean`, also contributed to the model's classification decisions. Overall, the results demonstrate that traffic timing, packet sizes, and flow behavior played a major role in distinguishing malicious traffic from normal traffic within the CICIDS2017 dataset.

Data Set 2: Preparation

```
***** CLEANING TRAIN DATA *****
Train Attack Categories:
attack_cat
Normal      37000
Generic     18871
Exploits    11132
Fuzzers     6062
DoS         4089
Reconnaissance 3496
Analysis    677
Backdoor    583
Shellcode   378
Worms       44
Name: count, dtype: int64

Train Shape After Cleaning:
(53884, 43)

***** CLEANING TEST DATA *****
Test Attack Categories:
attack_cat
Normal      56000
Generic     40000
Exploits    33393
Fuzzers     18184
DoS         12264
Reconnaissance 10491
Analysis    2000
Backdoor    1746
Shellcode   1133
Worms       130
Name: count, dtype: int64

Test Shape After Cleaning:
(100881, 43)
```

For this part of the project, the original UNSW-NB15 training and testing datasets provided with the dataset were used instead of creating a separate train/test split manually. Using the original datasets helped keep the intended separation between training data and unseen testing data so the models could be tested in a more realistic cybersecurity setting. During cleaning, extra spaces were removed from column names, infinity values and blank cells were converted into missing values, and rows with missing labels were removed. Duplicate rows were also removed to reduce repeated network traffic records. Since the datasets had already been cleaned previously, the additional preprocessing mainly focused on making sure the feature columns were converted into numeric values that could be used by Scikit-learn's DecisionTreeClassifier. Columns like id and attack_cat were removed because they were descriptive rather than useful for prediction. After preprocessing, the datasets were separated into feature matrices (X_train, X_test) and label vectors (y_train, y_test) for supervised machine learning classification.

After preprocessing and cleaning, the UNSW-NB15 dataset contained 42 numeric feature columns that were used for machine learning classification. The original UNSW training and testing datasets were kept separate in order to evaluate how well the decision tree models performed on unseen network traffic data. The training dataset contained 53,884 records and was used to train the models to recognize patterns associated with benign and malicious traffic. The testing dataset contained 10,881 records and was reserved only for evaluating model performance after training. Both the training and testing labels contained the binary classes 0 and 1, representing normal and malicious network activity. Keeping the original dataset split helped create a more realistic cybersecurity testing environment and allowed the models to be evaluated on data they had not previously seen, which is important for measuring how well the models generalize to real-world intrusion detection scenarios.

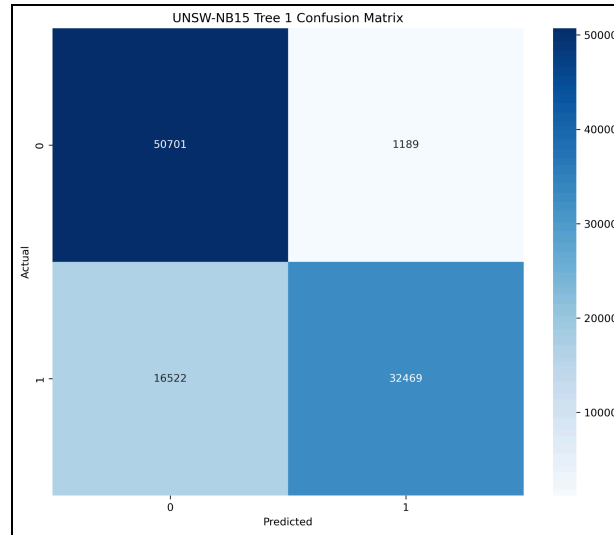
```
Training Feature Shape:
(53884, 42)

Testing Feature Shape:
(100881, 42)

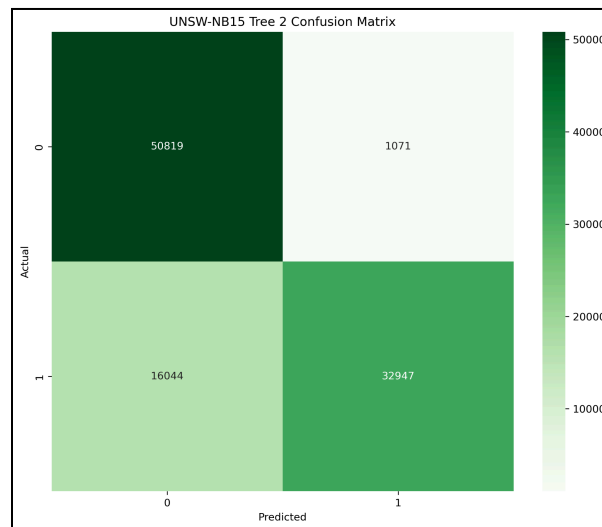
Training Labels:
['0' '1']

Testing Labels:
['0' '1']
```

Results



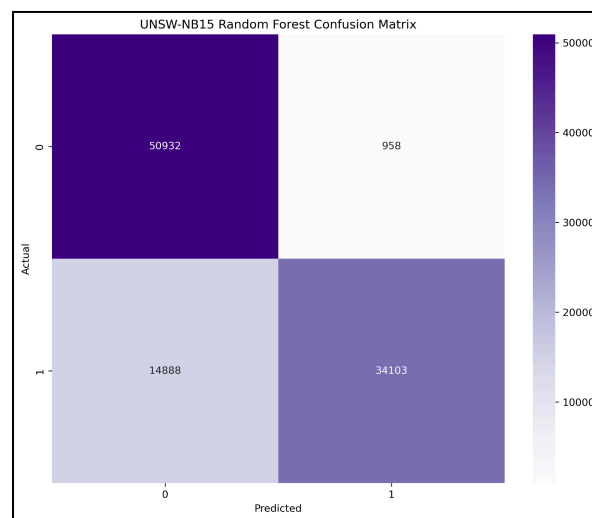
The first decision tree model achieved an overall accuracy of 82.44% when classifying network traffic in the UNSW-NB15 dataset as either benign or malicious. The confusion matrix shows that the model correctly classified 50,701 benign traffic records and 32,469 malicious traffic records. However, 16,522 malicious records were incorrectly classified as benign traffic, meaning some attacks were missed during classification. The model also produced 1,189 false positives where normal traffic was incorrectly flagged as malicious. The classification report shows that the model performed very well at identifying benign traffic with a recall of 98%, while the recall for malicious traffic was lower at 66%. This means the model was much stronger at recognizing normal traffic than detecting every attack. Overall, Tree 1 produced strong classification results while still keeping a simpler and easier-to-read tree structure.



The second decision tree model slightly improved the overall results with an accuracy of 83.03%. The confusion matrix shows that the model correctly classified 50,819 benign traffic

records and 32,947 malicious traffic records. Compared to Tree 1, the second model slightly reduced both false positives and false negatives, which improved the overall detection performance. The classification report shows that malicious traffic achieved a precision of 97%, meaning predictions labeled as malicious were usually correct. The recall for malicious traffic also increased slightly from 66% to 67%, showing a small improvement in attack detection. Tree 2 used the entropy criterion and a deeper tree structure, allowing the model to capture more detailed traffic patterns within the dataset. Overall, Tree 2 performed slightly better than Tree 1 and showed a stronger balance between detecting attacks and maintaining classification accuracy.

Random Forest



The Random Forest model achieved the strongest overall performance out of the three models with an accuracy of 84.29%. The confusion matrix shows that the model correctly classified 50,932 benign traffic records and 34,103 malicious traffic records. Compared to both decision tree models, the Random Forest model reduced the number of false positives and false negatives, showing better overall classification performance. The model incorrectly classified 958 benign records as malicious traffic and missed 14,888 malicious records by predicting them as benign. The classification report shows that the model achieved a precision of 97% for malicious traffic, meaning predictions labeled as attacks were usually correct. The recall for malicious traffic also increased to 70%, which was higher than both Tree 1 and Tree 2, showing that the Random Forest model detected more attacks overall. Since Random Forest combines predictions from multiple decision trees instead of relying on a single tree, it was able to capture more complex traffic patterns and improve overall reliability. Overall, the Random Forest model produced the best balance between accuracy, attack detection, and classification performance within the UNSW-NB15 dataset.

Discussion of Results

The analysis used both the CICIDS2017 and UNSW-NB15 intrusion detection datasets to evaluate how well machine learning models could classify network traffic as either benign or malicious. Decision Tree Classification models and a Random Forest model were trained and compared using traffic-flow features such as packet lengths, packet rates, timing behavior, and connection activity. Across both datasets, the Random Forest model consistently produced the strongest overall performance. For the CICIDS2017 dataset, the Random Forest model achieved the highest accuracy at 99.44%, outperforming the individual decision tree models while reducing false positives and false negatives. The UNSW-NB15 dataset was more difficult to classify accurately, but the Random Forest model still produced the best balance between attack detection and overall classification performance compared to the decision tree models. The feature importance analysis also showed that packet lengths, traffic timing behavior, packet rates, and connection activity were some of the most important factors used to identify malicious traffic within the datasets.

The results showed that Random Forest worked best because it combined predictions from multiple decision trees instead of relying on a single tree structure. This helped reduce overfitting and allowed the model to recognize more detailed and complex traffic patterns within the datasets. While the individual decision tree models were easier to visualize and interpret, they missed more malicious traffic compared to the Random Forest model. Overall, the analysis demonstrated that supervised machine learning models can effectively support cybersecurity intrusion detection by automatically identifying suspicious network activity with high accuracy. The findings suggest that machine learning-based intrusion detection systems could help improve cybersecurity monitoring, reduce security risks, and assist analysts in detecting malicious activity before attacks cause larger damage.

Classification Models Part 2:

What's Naive Bayes?

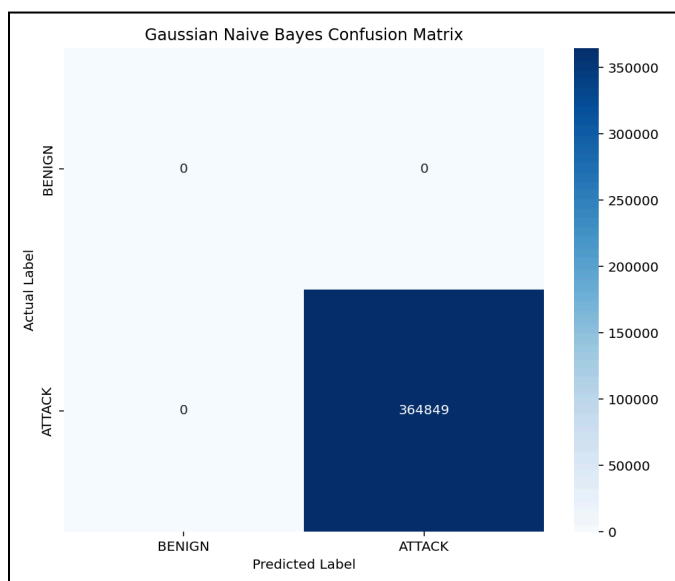
Naive Bayes is a machine learning classification algorithm that uses probability to predict whether data belongs to a specific category or class. The model is based on Bayes' Theorem and assumes that each feature contributes independently to the final prediction, which allows the algorithm to make fast and efficient classifications even when working with large datasets. In this project, Naive Bayes was used to classify network traffic as either normal traffic or malicious attack traffic using cybersecurity datasets such as UNSW NB15 and CICIDS 2017. The model analyzed features related to network behavior, including packet counts, traffic rates, connection statistics, and protocol information, to determine the probability that a network connection represented suspicious activity. Naive Bayes is useful for this project because it is

computationally efficient, handles high-dimensional cybersecurity data well, and can quickly detect patterns associated with cyberattacks. This makes it valuable for intrusion detection systems and real-time cybersecurity monitoring environments where organizations need fast threat detection to reduce the risk of breaches or network compromise.

Date Set 1: Naive Bayes Preparation

The CICIDS2017 dataset was loaded into Python using Pandas and cleaned before running the Naive Bayes models. During preprocessing, extra spaces were removed from the column names and the Label column was converted into binary values where BENIGN traffic became 0 and attack traffic became 1. Infinite values and missing values were cleaned by replacing them with 0 so the models could run correctly. The dataset was then split into features and target labels, and the categorical columns were separated from the numeric columns for preprocessing. Gaussian Naive Bayes used one hot encoded categorical features while keeping the continuous numeric values unchanged, since GaussianNB works best with continuous data. Multinomial Naive Bayes also used one hot encoding, but the numeric values were scaled using MinMaxScaler because MultinomialNB requires nonnegative values. During development, an error occurred stating "Number of classes, 1, does not match size of target_names, 2" because only one class was being detected after preprocessing. This was fixed by cleaning the label values using `.str.strip()` and `.str.upper()` to make sure all BENIGN labels were formatted consistently before binary conversion. The `classification_report()` and `confusion_matrix()` functions were also updated with `labels=[0,1]` so both benign and attack classes would always be included in the evaluation results.

Gaussian NB



Gaussian Naive Bayes was chosen for the CICIDS2017 dataset because the dataset contains a large number of continuous numeric network traffic features such as flow duration, packet lengths, byte transfers, and traffic statistics.

GaussianNB works well with continuous numerical data because it assumes the features follow a Gaussian, or normal, distribution. To prepare the dataset for the model, categorical features were converted using encoding while the numeric features were left in their continuous form. The goal of the model was to classify network traffic as either BENIGN traffic or ATTACK traffic based

on the network behavior patterns in the dataset. Gaussian Naive Bayes was also chosen because

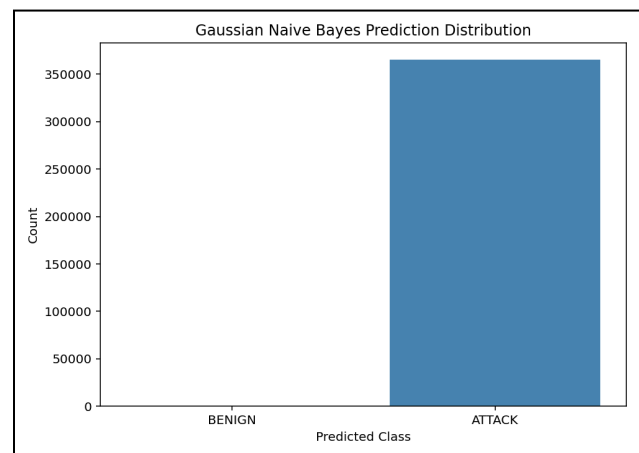
it is computationally efficient, easy to implement, and commonly used as a baseline machine learning model for intrusion detection and anomaly detection systems.

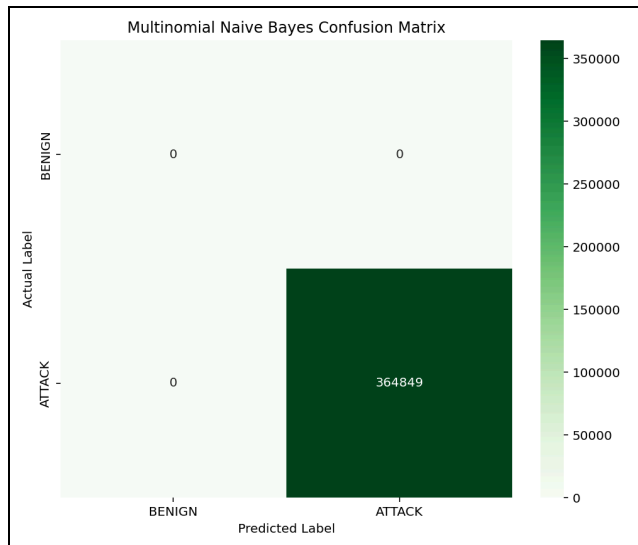
The confusion matrix and prediction distribution results showed that the Gaussian Naive Bayes model predicted every testing record as an attack. The confusion matrix displayed 364,849 correctly predicted attack records while all benign traffic predictions remained 0. This means the model achieved perfect detection for attack traffic in this specific testing set, but it completely failed to identify any benign traffic records. The prediction distribution chart also confirmed this behavior because the entire prediction count appeared under the ATTACK category while the BENIGN category had no predictions. These results suggest that the dataset being tested likely contained only attack traffic after preprocessing or that the model became heavily biased toward predicting attacks due to class imbalance within the dataset.

The probability outputs from the Gaussian Naive Bayes model demonstrated extremely high confidence in attack predictions because the model classified every testing record as an ATTACK. This is why the confusion matrix showed 0 predictions for the BENIGN category while the ATTACK category contained the very large value of 364,849. The model became heavily biased toward predicting attacks, likely because the testing dataset contained mostly attack traffic or because the class imbalance caused the learned probability distributions to strongly favor the attack class. As a result, the probability scores consistently assigned much higher probabilities to ATTACK than to BENIGN, causing all traffic to be classified as malicious. While this behavior can be useful for aggressively detecting suspicious activity, it also highlights an important limitation because the model failed to recognize any legitimate traffic. In a realworld cybersecurity environment, this would likely create excessive alerts and make it more difficult for analysts to separate actual threats from normal network activity.

Multinomial NB

Multinomial Naive Bayes was chosen for the CICIDS2017 dataset because the model works well with nonnegative feature values and highdimensional datasets that contain many encoded categorical and scaled numeric features. To prepare the dataset, categorical features such as the dataset day values were converted using one hot encoding while the numeric network traffic features were scaled using MinMaxScaler so all values remained between 0 and 1. After preprocessing, the dataset expanded to 58 feature columns, allowing the model to learn patterns across multiple traffic characteristics. MultinomialNB was selected because it is commonly used for probability based classification problems and performs efficiently on large datasets with many transformed features.

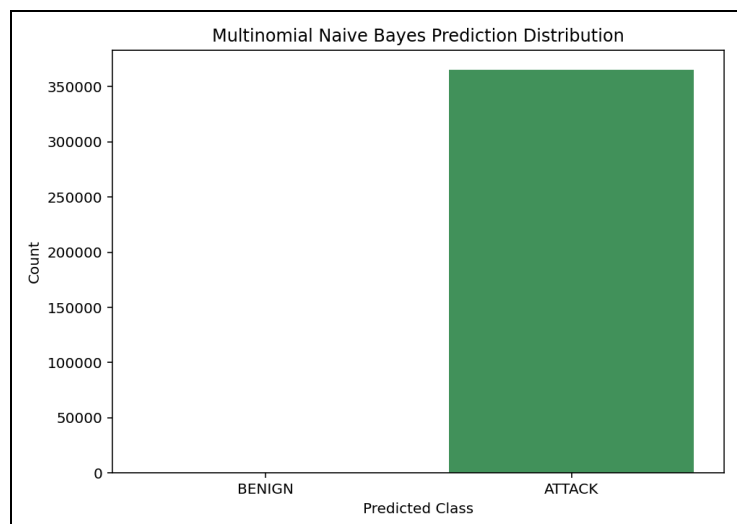




The confusion matrix and prediction distribution results showed that the Multinomial Naive Bayes model classified every testing record as an ATTACK. This is why the confusion matrix contained 0 values across the entire BENIGN row while the ATTACK category contained the very large value of 364,849. The prediction distribution chart also confirmed that every prediction made by the model fell under the ATTACK category. Because the testing dataset did not contain any benign traffic after preprocessing, the model achieved an accuracy score of 1.0, or 100% accuracy, since every testing record belonged to the attack class and every prediction matched

correctly. While this produced perfect accuracy mathematically, it does not necessarily mean the model would perform perfectly in a real world cybersecurity environment because it was never tested against legitimate benign traffic during evaluation.

The probability prediction outputs also showed extremely high confidence in the attack classifications. Every probability example produced $P(\text{ATTACK}) = 1.0$, meaning the model was completely certain that each network traffic record represented malicious activity. This happened because the model only learned and predicted one dominant class during testing, causing the attack probability distribution to overpower any possibility of benign predictions. While highly confident predictions



may appear desirable for intrusion detection systems, the absence of benign predictions highlights a major limitation because the model was not able to demonstrate whether it could properly separate normal traffic from malicious traffic. In a realworld cybersecurity system, a model that predicts only attacks could generate overwhelming alert volumes and reduce the effectiveness of human analysts who must investigate and prioritize security events.

Naive Bayes Results

The results from the Gaussian Naive Bayes and Multinomial Naive Bayes models showed that the CICIDS2017 dataset was heavily dominated by attack traffic during testing, causing

both models to classify every record as an ATTACK. From a cybersecurity perspective, this means the system was extremely aggressive at detecting suspicious activity and would likely identify threats very quickly before major damage could occur. However, the results also highlighted an important limitation because neither model was able to recognize legitimate benign traffic during testing. This creates a poor balance between false positives and false negatives since a realworld intrusion detection system must be able to distinguish between normal traffic and actual attacks. If every event is flagged as malicious, cybersecurity analysts could become overwhelmed with alerts, making it harder to prioritize real threats effectively. The results also suggest that more obvious and high volume attack behaviors were easier for the models to detect, while the models were not properly evaluated against more subtle or normal traffic patterns due to the class imbalance in the testing dataset.

From a machine learning perspective, the results demonstrated that Naive Bayes models can quickly learn dominant traffic patterns within large cybersecurity datasets, but they may become biased when the dataset is heavily imbalanced. Both models achieved perfect accuracy mathematically because all testing samples belonged to the attack class, but this does not necessarily indicate strong generalization to realworld environments where both benign and malicious traffic coexist. The results also show why additional techniques such as unsupervised learning, clustering, or PCA could be useful for cybersecurity anomaly detection. PCA and dimensionality reduction could help simplify complex network traffic features and improve the models' ability to separate meaningful attack behaviors from legitimate activity. In a realworld cybersecurity workflow, systems like these could be integrated into SIEM or network monitoring platforms to automatically flag suspicious traffic and assist analysts with real time threat monitoring. The probability outputs, confusion matrices, and prediction visualizations also provide interpretable results that analysts could use to understand how confident the models are when identifying potential attacks.

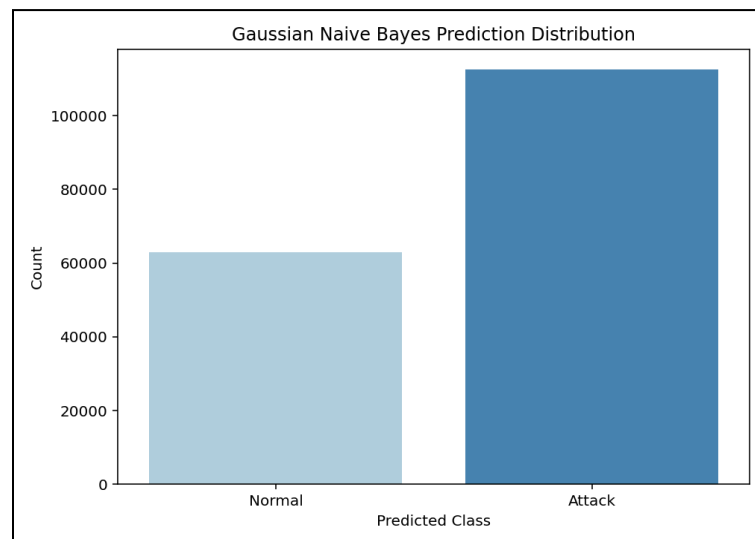
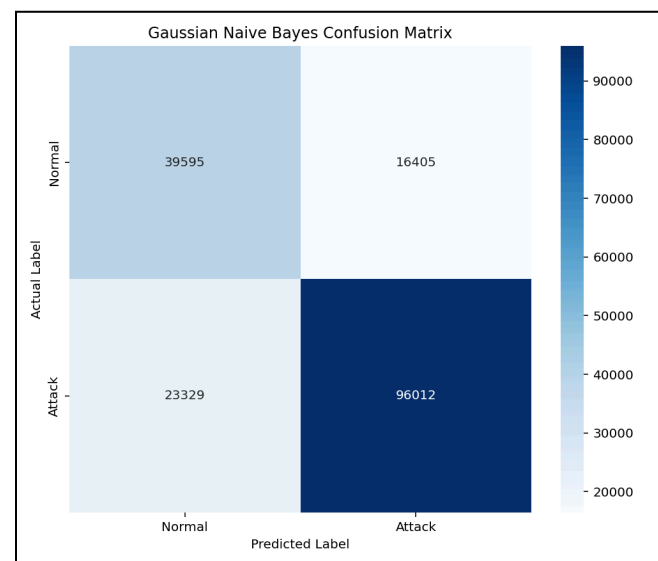
Date Set 2:Naive Bayes Preparation

The UNSW-NB15 training and testing datasets were first loaded into Python using Pandas. Before cleaning, the training dataset contained 82,332 rows and 45 columns, while the testing dataset contained 175,341 rows and 45 columns. During the cleaning process, the id and attack_cat columns were removed because they were not necessary for binary classification and could interfere with the Naive Bayes models. Any missing values were replaced with 0 to avoid errors during preprocessing and model training. After cleaning, both datasets contained 43 columns. The cleaned datasets included three categorical columns (proto, service, and state) along with thirty-nine numeric columns related to network traffic behavior such as packet counts, byte transfers, rates, TCP statistics, and connection metrics. The data was then separated into features and the target label column, where 0 represented normal traffic and 1 represented attack traffic. To prepare the datasets for Naive Bayes classification, categorical variables were converted using one-hot encoding. The Gaussian Naive Bayes model used the continuous numeric values directly since it works best with continuous data, while the Multinomial Naive Bayes model used MinMaxScaler to convert all numeric values into non-negative values because MultinomialNB requires non-negative feature data.

Gaussian NB

Gaussian Naive Bayes was chosen for this project because the UNSW-NB15 dataset contains many continuous numeric features related to network traffic behavior, including packet counts, byte transfers, connection durations, load rates, TCP timing statistics, and other quantitative network measurements. GaussianNB works best with continuous numerical data and assumes that the features follow a Gaussian, or normal, distribution. To prepare the dataset for this model, the categorical columns (proto, service, and state) were converted into numeric values using one-hot encoding while the numeric features were kept in their continuous form. The goal of the model was to classify network traffic as either normal traffic (0) or attack traffic (1) based on the network connection information provided in the dataset.

The Gaussian Naive Bayes model achieved an overall accuracy of approximately 77.34%, showing that the model was able to correctly classify most of the network traffic records. The classification report showed stronger performance for detecting attack traffic, with a precision of 0.85 and recall of 0.80 for attack records. This indicates that the model performed better at identifying malicious traffic than identifying normal traffic. The confusion matrix further supports these results. Out of 56,000 actual normal traffic records, the model correctly classified 39,595 as normal while incorrectly classifying 16,405 as attacks. For attack traffic, the model correctly identified 96,012 attack records while incorrectly labeling 23,329 attacks as normal traffic. The confusion matrix heatmap visualization also showed that the darkest and largest section appeared in the correctly predicted attack category, indicating that the model was highly effective at detecting attack



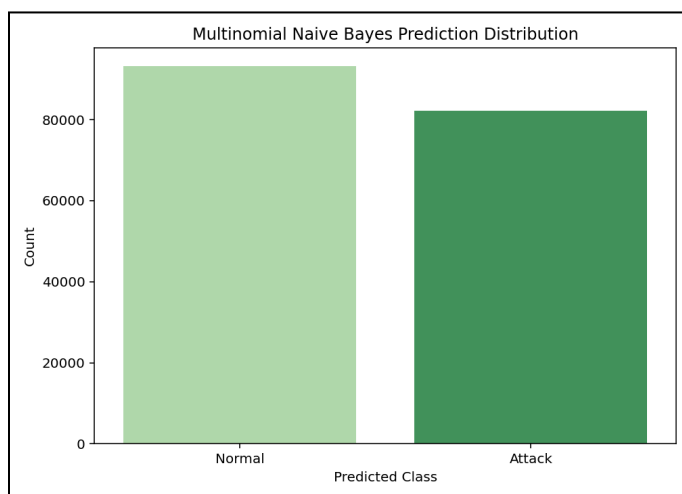
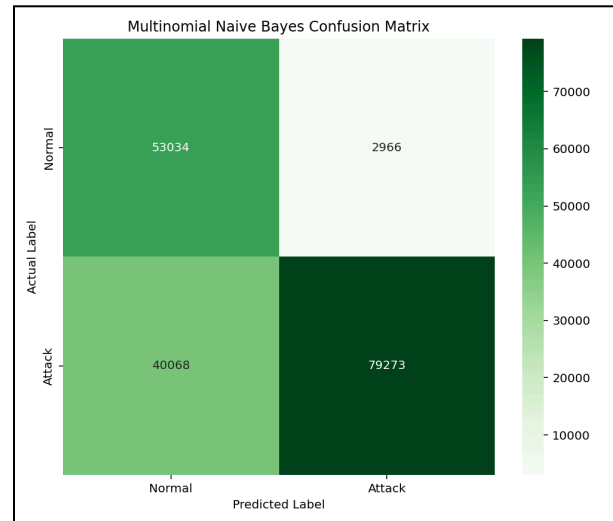
traffic overall. Additionally, the prediction distribution chart showed that the model predicted more attack records than normal records, which aligns with the larger number of attack samples present in the testing dataset.

The probability examples demonstrated how confident the Gaussian Naive Bayes model was for each prediction. For example, one record had a $P(\text{Attack})$

probability of 0.856498, meaning the model was very confident that the traffic represented an attack. Another prediction showed $P(\text{Normal})$ equal to 0.997200, indicating an extremely high confidence that the traffic was normal. Some predictions were much closer to 0.50, such as $P(\text{Normal}) = 0.505567$ and $P(\text{Attack}) = 0.494433$, showing situations where the model was uncertain because the traffic characteristics shared similarities between normal and malicious behavior. Overall, the probability outputs helped illustrate not only the final classification decision, but also how certain the model was when making each prediction.

Multinomial NB

Multinomial Naive Bayes was chosen because the UNSW-NB15 dataset contains a combination of categorical features and numeric network traffic data that could be transformed into non-negative feature values. MultinomialNB works best with discrete or non-negative quantitative data, so the categorical variables (proto, service, and state) were converted using one-hot encoding while the numeric features were scaled using MinMaxScaler to ensure all values remained non-negative. After preprocessing, the formatted dataset expanded to 190 columns because each categorical value became its own encoded feature column. The Multinomial Naive Bayes model was then trained to classify network traffic as either normal traffic (0) or attack traffic (1) using the transformed dataset.



The Multinomial Naive Bayes model achieved an overall accuracy of approximately 75.46%, which was slightly lower than the Gaussian Naive Bayes model. However, the classification report showed very strong precision for attack traffic, with a precision score of 0.96, meaning that when the model predicted an attack, it was usually correct. The recall for normal traffic was also very high at 0.95, showing that the model was very effective at identifying legitimate traffic records. The confusion matrix showed that the model correctly classified 53,034

normal traffic records and 79,273 attack records. Only 2,966 normal records were incorrectly classified as attacks, which was significantly lower than the Gaussian Naive Bayes model. However, the model struggled more with detecting all attack traffic records, incorrectly labeling 40,068 attack records as normal traffic. The confusion matrix heatmap visually showed that the

strongest predictions occurred in the correctly predicted normal and attack categories, while the prediction distribution chart showed that the model predicted slightly more normal traffic than attack traffic overall.

The probability examples also demonstrated how confident the Multinomial Naive Bayes model was during classification. Many of the predictions showed very high confidence for normal traffic, such as $P(\text{Normal}) = 0.987684$ and $P(\text{Attack}) = 0.012316$, indicating that the model was extremely certain those records represented legitimate network activity. Other examples, such as $P(\text{Normal}) = 0.701251$ and $P(\text{Attack}) = 0.298749$, showed cases where the model was less certain because the traffic shared characteristics of both normal and malicious behavior. Compared to Gaussian Naive Bayes, the Multinomial model generally produced more confident probability scores for normal traffic predictions. Overall, the probability outputs helped explain not only the final predicted class, but also the confidence level behind each classification decision.

Naive Bayes Results

The results from both the Gaussian Naive Bayes and Multinomial Naive Bayes models demonstrate that machine learning can be effective for detecting malicious network traffic within a cybersecurity environment. Both models were able to identify a large number of attack records, showing that anomaly detection systems can help organizations detect cyber threats early enough to reduce the likelihood of serious damage or compromise. The Gaussian Naive Bayes model performed slightly better overall in terms of accuracy and attack detection recall, meaning it was more successful at finding malicious traffic. However, it also produced a larger number of false positives by incorrectly labeling some normal traffic as attacks. In a real-world cybersecurity setting, this trade-off is important because too many false positives could overwhelm analysts with unnecessary alerts, while too many false negatives could allow real attacks to go unnoticed. The Multinomial Naive Bayes model showed fewer false positives, meaning it was more conservative when flagging attacks, but it also missed a larger number of actual attack records. These results highlight the challenge of balancing security effectiveness with practical usability in intrusion detection systems.

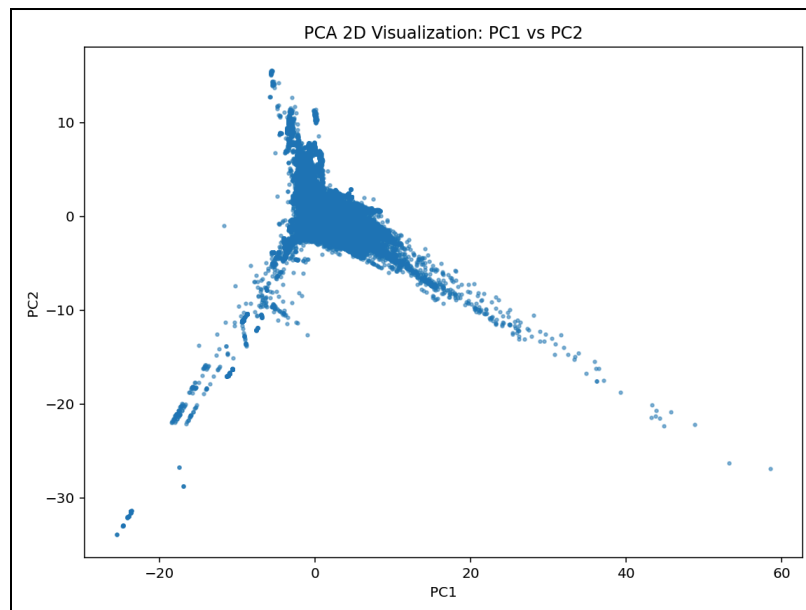
The results also showed that certain attack patterns were easier for the models to detect than others. Stronger and more obvious attack behaviors were often classified correctly with high probability scores, while traffic that closely resembled normal behavior was more difficult for the models to distinguish. The probability outputs demonstrated this uncertainty, with some predictions producing confidence values close to 50%, indicating that the model recognized overlapping characteristics between normal and malicious traffic. From a machine learning perspective, both models generalized reasonably well to the unseen testing dataset, suggesting that the trained classifiers could still perform effectively on new network traffic data. In a real-world environment, systems like these could be integrated into SIEM or network monitoring platforms to support automated threat detection and assist cybersecurity analysts in prioritizing suspicious events for investigation. The confusion matrices and prediction probability outputs also provide analysts with understandable and actionable information that could help support faster decision-making during incident response and threat monitoring.

Analyses

Phase 1

The clustering and anomaly visualization results demonstrate that the dataset contains multiple distinct patterns of network behavior, with clear separation between common activity and less frequent observations. In the anomaly plot, most data points are concentrated in dense central clusters, representing typical system behavior such as normal traffic flows and routine operations. In contrast, a smaller number of points extend outward and appear more scattered, indicating observations that differ significantly from the majority. These patterns suggest that the clustering model is effective at identifying both dominant behaviors and deviations, which is essential for analyzing large-scale network traffic data.

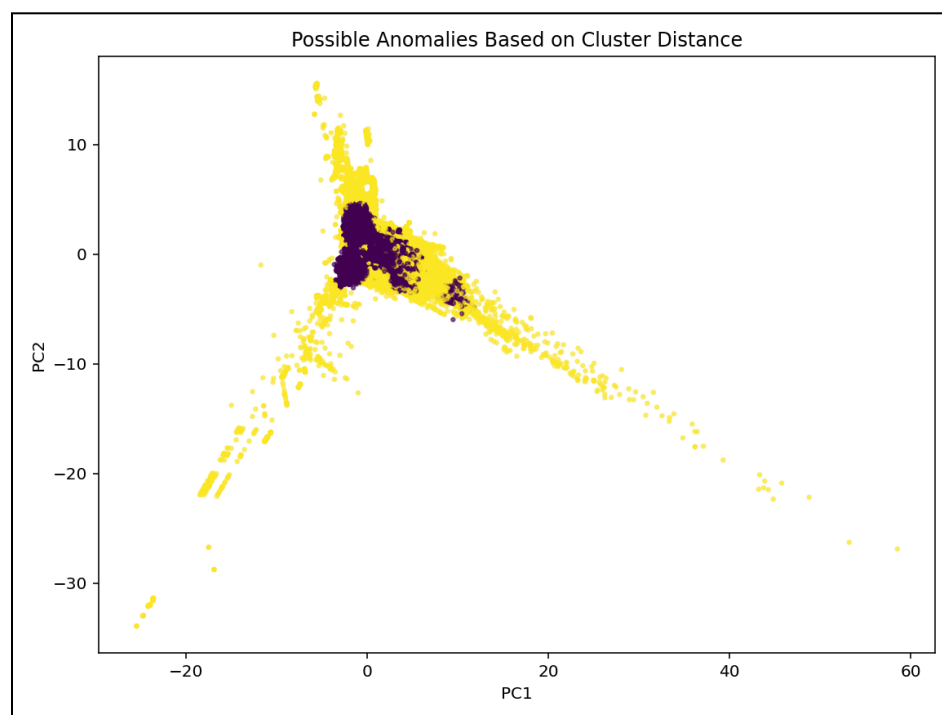
The use of Principal Component Analysis (PCA) was critical in achieving this result. By reducing the high-dimensional dataset into three principal components, PCA preserves the most important variance while removing noise and redundancy. Although only a portion of the total variance is retained, the transformation enhances the separation between clusters and makes the data easier to visualize and interpret. This improvement is reflected in the higher silhouette scores observed after PCA, indicating stronger cluster cohesion and clearer boundaries. As a result, PCA enables more accurate grouping of similar behaviors and improves the overall effectiveness of clustering.



Several key insights can be drawn from these results. First, the dataset is not uniform and contains multiple behavioral patterns, indicating a mix of normal and potentially abnormal activity. Second, increasing the number of clusters allows the model to capture more detailed structure in the data, with $k = 7$ providing the best balance between separation and granularity.

The presence of smaller clusters and scattered points suggests that rare or unusual events exist within the dataset. These findings show that unsupervised learning can uncover meaningful patterns without relying on labeled data, making it well-suited for exploratory analysis in cybersecurity contexts.

In relation to threat detection, these results demonstrate that clustering can be used to separate normal behavior from suspicious or anomalous activity. Dense clusters (dark purple) represent expected system behavior, while points located farther from cluster centers indicate potential anomalies. These anomalies may correspond to unusual network events such as irregular traffic patterns, abnormal usage behavior, or potential attack activity. Because the model identifies these patterns based on deviation rather than predefined rules, it is capable of detecting new or evolving threats that may not match known signatures.



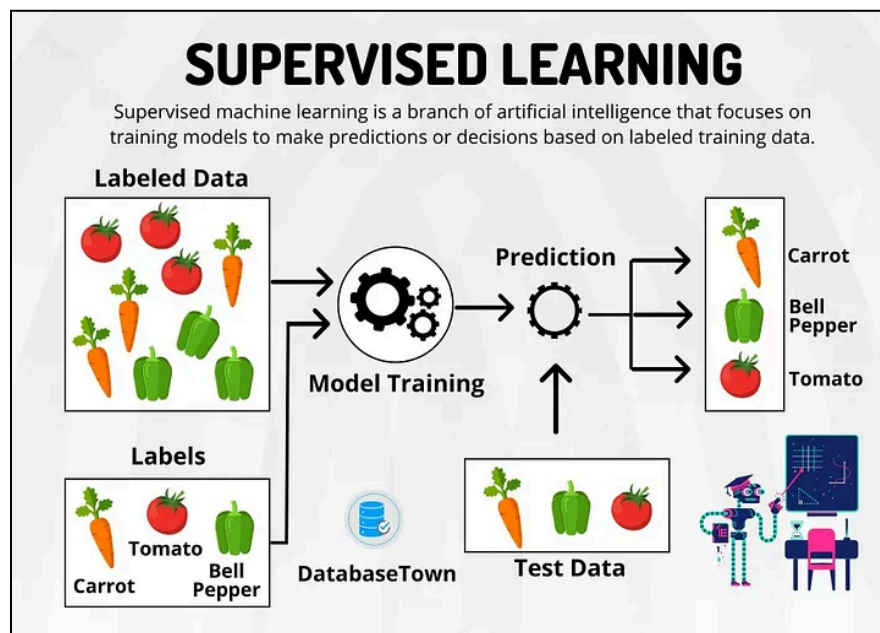
The anomaly detection process further strengthens this analysis by quantifying how far each data point deviates from its assigned cluster. By selecting the top 5% of points with the greatest distance from cluster centroids, the model highlights the most unusual observations in the dataset. In the visualization, these anomalies appear as points that lie outside the dense cluster regions or along extended branches of the distribution. While not all anomalies represent malicious activity, they provide a focused set of data points for further investigation. This approach allows analysts to prioritize potential threats, improving efficiency and enabling faster response times. Overall, these results demonstrate that clustering combined with PCA and anomaly detection provides a powerful framework for identifying hidden patterns and supporting proactive cybersecurity analysis.

In relation to the research questions, the results support Q1 and Q4 by showing that the system can identify anomalous behavior early and without prior knowledge of specific attack

types, indicating potential for detecting evolving threats. The improved clustering performance after PCA also addresses Q7, demonstrating that dimensionality reduction enhances anomaly detection by making patterns clearer. Additionally, the identification of smaller, isolated clusters provides insight into Q3, suggesting that more extreme or distinct attacks are easier to detect, while subtle anomalies remain more challenging. Overall, these results highlight that clustering-based approaches can support real-world cybersecurity workflows (Q8 and Q9) by generating interpretable outputs that help analysts detect, investigate, and respond to potential threats more efficiently. These questions will be further explored and more fully answered in the final analysis as the results are redefined and evaluated

Phase 2:

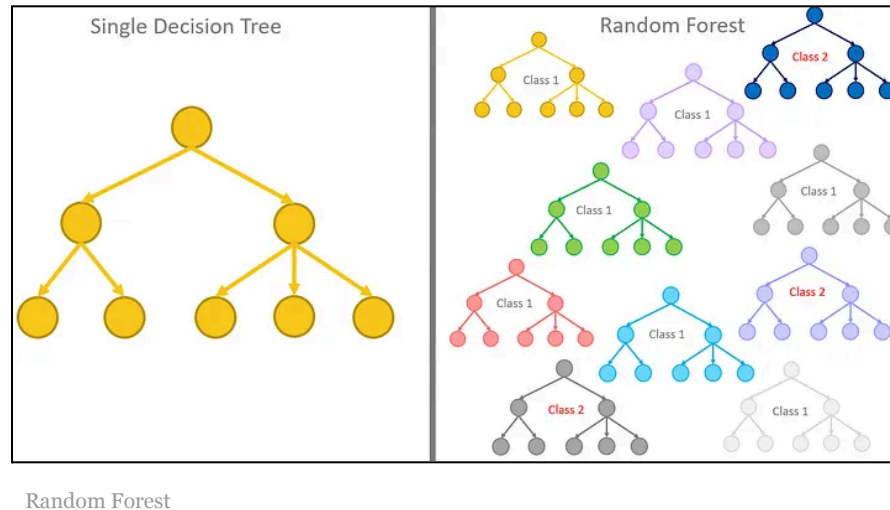
The second phase of the project focused on applying supervised machine learning techniques to determine how effectively different models could identify malicious network activity. Unlike the unsupervised learning methods used in Phase 1, which searched for patterns without knowing whether traffic was normal or malicious, the models in this phase were trained using labeled data. This allowed the algorithms to learn the characteristics associated with both benign and attack traffic and then use those learned patterns to classify previously unseen network activity. The primary goal of this phase was to compare multiple machine learning approaches and determine which models provided the most accurate and reliable results for cybersecurity anomaly detection.



The first models explored during this phase were Linear Regression and Logistic Regression. Linear Regression was included as a baseline model to investigate whether network traffic characteristics could be used to predict network behavior through a continuous mathematical relationship. The results demonstrated that network traffic patterns are significantly more complex than a simple linear relationship can capture. The low R^2 values and

large prediction errors observed across both datasets indicate that network activity is influenced by multiple interacting variables that cannot be effectively modeled using a basic linear approach. While the model provided useful insight into the structure of the data, it was not well suited for anomaly detection or intrusion detection tasks.

In contrast, Logistic Regression produced substantially stronger results because it approached the problem as a classification task rather than a prediction problem. Instead of attempting to predict exact traffic values, the model focused on determining whether traffic should be classified as benign or malicious. The confusion matrices showed that the model successfully identified most attack traffic while maintaining relatively low classification error rates. These findings suggest that cybersecurity anomaly detection is better approached as a classification problem, where the goal is to separate suspicious activity from legitimate activity, rather than attempting to predict exact network measurements. Logistic Regression also demonstrated the practical value of simpler machine learning models, providing strong performance while remaining computationally efficient and easy to interpret.



The project then expanded to Decision Tree and Random Forest classification models. Decision Trees provided a highly interpretable method for classifying network traffic because every prediction could be traced through a series of feature-based decision rules. This transparency is particularly valuable in cybersecurity environments where analysts often need to understand why an alert was generated. The decision tree visualizations showed that traffic features related to packet lengths, traffic timing, and flow behavior played important roles in separating benign and malicious activity. While both decision tree configurations produced strong results, the deeper tree structure was more effective at detecting malicious traffic because it captured more detailed relationships within the datasets.

Random Forest further improved upon the Decision Tree approach by combining predictions from multiple trees rather than relying on a single decision structure. This ensemble learning strategy reduced overfitting and improved the model's ability to generalize to unseen data. Across both the CICIDS2017 and UNSW-NB15 datasets, Random Forest consistently produced the highest accuracy and strongest overall detection performance. The feature importance analysis also provided valuable insight into which traffic characteristics contributed

most significantly to attack detection. Features related to packet lengths, flow timing, connection rates, and packet behavior consistently appeared among the most influential variables. These results demonstrate that malicious network activity often leaves identifiable patterns that machine learning algorithms can successfully recognize even when operating across different datasets and network environments.

The final models evaluated during Phase 2 were Gaussian Naive Bayes and Multinomial Naive Bayes. These models approached anomaly detection using probability-based classification rather than decision boundaries or tree structures. The results demonstrated both strengths and limitations of the Naive Bayes approach. On the UNSW-NB15 dataset, both models were able to identify a substantial amount of attack traffic, although each exhibited different tradeoffs between false positives and false negatives. However, the CICIDS2017 results revealed how heavily probability-based models can be influenced by class imbalance. In several cases, the models became overly biased toward predicting attack traffic, which reduced their ability to distinguish legitimate network behavior from malicious activity. While this aggressive detection behavior can help reduce missed attacks, it also increases the likelihood of generating excessive alerts that may overwhelm analysts.

Taken together, the results from Phase 2 provide strong evidence that supervised machine learning can effectively support cybersecurity anomaly detection. The comparison between models demonstrated that more advanced classification approaches generally outperformed simpler regression methods, while ensemble techniques such as Random Forest provided the most reliable overall performance. The findings also highlight the importance of balancing detection accuracy, interpretability, and computational efficiency when selecting machine learning solutions for real-world cybersecurity environments. Overall, this phase demonstrated that machine learning models can successfully learn patterns associated with malicious behavior and use those patterns to assist analysts in identifying threats, prioritizing investigations, and improving organizational cybersecurity defenses.

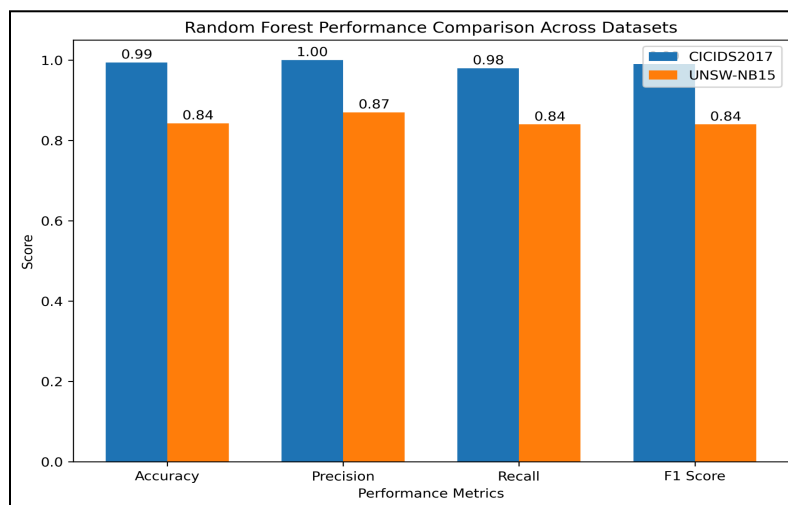
Results

Performance Comparison Across All Models

Model	Dataset	Accuracy	Precision	Recall	F1
Random Forest	CICIDS2017	99.44%	100%	97%	98%
Decision Tree 2	CICIDS2017	98.89%	94%	99%	96%
Logistic Regression	CICIDS2017	89.07%	63%	87%	73%
Random Forest	UNSW-NB15	84.29%	97%	70%	81%
Decision Tree 2	UNSW-NB15	83.03%	97%	67%	79%
Logistic Regression	UNSW-NB15	81.22%	85%	86%	85%
Gaussian NB	UNSW-NB15	77.34%	85%	80%	82%
Multinomial NB	UNSW-NB15	75.46%	96%	66%	78%

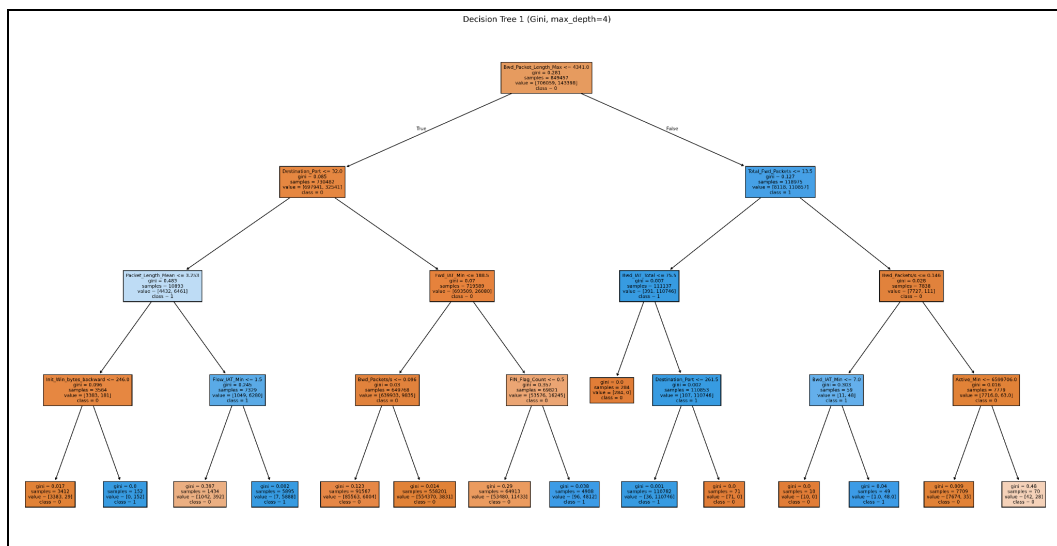
The results showed that not all machine learning techniques are equally effective for anomaly detection. The supervised learning models generally performed the best because they were trained using labeled examples of normal and malicious traffic. Random Forest stood out as the strongest model overall, producing the highest accuracy and the most balanced detection results on both datasets. While Decision Trees and Logistic Regression were still able to identify a large amount of malicious traffic, they were more likely to make classification errors. Random Forest appeared to benefit from using multiple decision trees together, which helped it recognize more complex patterns in the data. Since it performed well on both CICIDS2017 and UNSW-NB15, the model appears to be more adaptable and reliable than the other approaches tested during this project.

Random Forest Results



Random Forest demonstrated the strongest overall performance among all supervised learning models evaluated in this study. On the CICIDS2017 dataset, Random Forest achieved an accuracy of 99.44%, with a precision of 1.00, a recall of 0.98, and F1 score of 0.99, indicating near-perfect classification performance. Similarly, on the UNSW-NB15 dataset, Random Forest outperformed both decision tree models, achieving an accuracy of 84.29%, a precision of 0.87, a recall of 0.84, and F1 score of 0.84. By combining the predictions of multiple decision trees, the model reduced overfitting while capturing complex patterns within network traffic data. The consistently high performance across both datasets demonstrates Random Forest's effectiveness in distinguishing between benign and malicious network activity. Although Random Forest produced the best results, it offers less interpretability than a single decision tree because predictions are generated from an ensemble of trees rather than a single set of decision rules. In a real-world cybersecurity environment, Random Forest would be a strong candidate for deployment within intrusion detection systems due to its ability to accurately identify attacks while minimizing classification errors. Its robust performance across two different network intrusion datasets highlights its value for monitoring large-scale networks and supporting cybersecurity analysts in threat detection and response.

Decision Tree Results

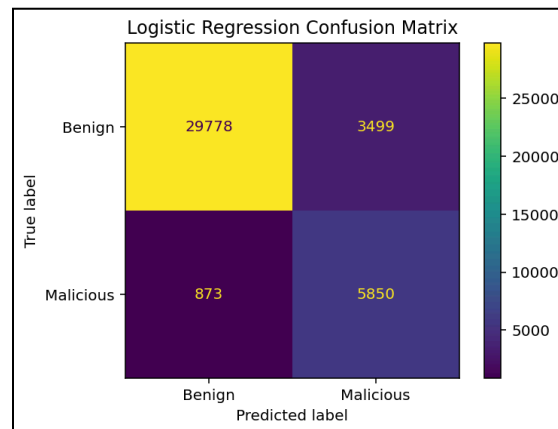
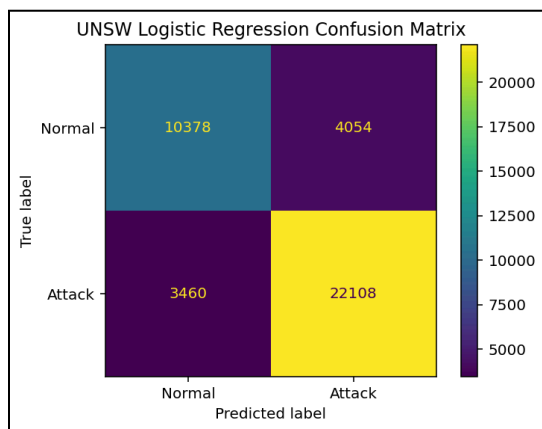


The Decision Tree models performed well overall, especially when tested on the CICIDS2017 dataset. Tree 1 achieved an accuracy of 97.43%, while Tree 2 improved to 98.89%, making it the stronger of the two configurations. The improvement suggests that Tree 2 was better at identifying malicious traffic while reducing the number of classification errors. One advantage of decision trees is that they are easy to understand and interpret. Unlike some machine learning models that produce predictions without showing how a decision was reached, decision trees provide a clear path of rules that can be followed step by step. In this project, features such as Bwd_Packet_Length_Max, Destination_Port, Packet_Length_Mean, and Total_Fwd_Packets were frequently used to separate normal traffic from attacks. Both trees used Bwd_Packet_Length_Max as the root node, indicating that it was one of the most important features in the classification process. This level of transparency is useful in cybersecurity because analysts can examine the decision path and understand why specific network traffic was flagged as suspicious. However, decision trees also have limitations. Since they rely on a single tree structure, they can become too closely fitted to the training data and may not perform as well on new data compared to ensemble methods such as Random Forest. Even with this limitation, decision trees remain valuable tools when explainability is important because they allow security professionals to clearly see the factors that influenced each classification decision.

Logistic Regression Results

Logistic Regression achieved decent results on both datasets and was able to identify most malicious traffic while keeping classification errors at a manageable level. For the CICIDS2017 dataset, the model correctly classified 29,778 benign connections and 5,850 malicious connections. It incorrectly labeled 3,499 benign connections as malicious and missed 873 malicious connections by classifying them as benign. These results show that the model was generally effective at detecting attacks, with a relatively low number of missed threats. On the UNSW-NB15 dataset, the model correctly identified 10,378 normal connections and 22,108

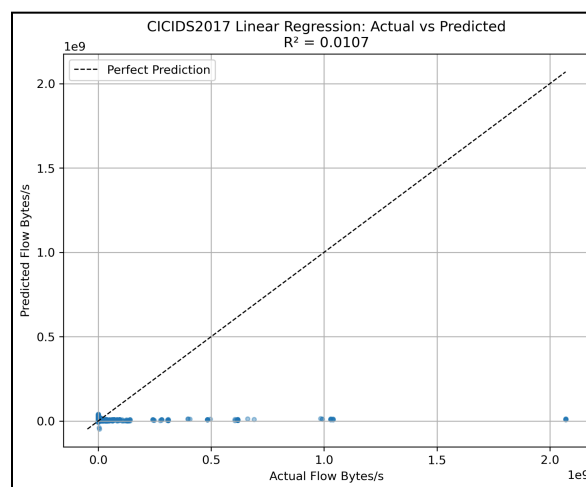
attack connections. However, it incorrectly classified 4,054 normal connections as attacks and 3,460 attacks as normal traffic. Compared to CICIDS2017, the UNSW-NB15 dataset resulted in more classification errors, suggesting that it was more difficult for the model to distinguish between normal and malicious traffic patterns.



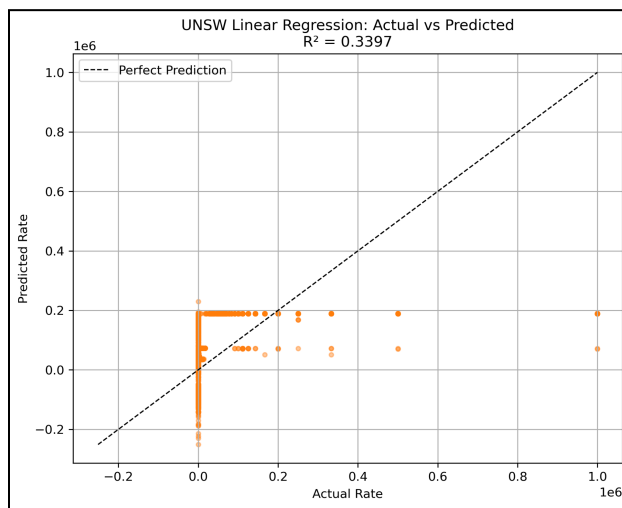
The confusion matrices show that Logistic Regression was able to learn useful patterns from the network traffic data, but it was not as effective as some of the other machine learning models tested. Because Logistic Regression uses a linear decision boundary, it can struggle when the relationship between features becomes more complex. Network traffic often contains nonlinear patterns that are difficult for a simple linear model to capture. This likely contributed to some attacks being classified as normal traffic and some legitimate traffic being flagged as malicious. Even with these limitations, Logistic Regression served as a useful baseline model and demonstrated that the available features contained enough information to separate normal and malicious traffic to a certain extent. While the model offers fast training and low computational cost, the results suggest that models such as Decision Trees and Random Forests are better suited for handling the complexity of modern network traffic and cyberattack behavior.

Linear Regression Results

Linear Regression was used to see if network traffic features could be used to predict traffic behavior, but the results showed that it was not very effective on either dataset. For the CICIDS2017 dataset, the model had an R^2 value of 0.0107, which means it explained only a very small amount of the variation in the data. The error values were also very high, with an RMSE of 34,019,764.23 and an MAE of 3,054,821.70. Looking at the scatterplot, most of the predicted values are clustered near the bottom of the graph and do not follow the



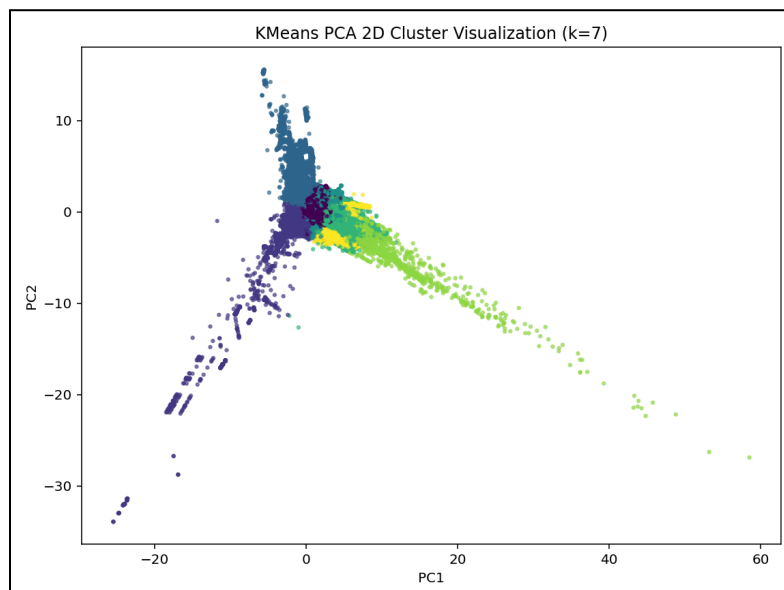
perfect prediction line. This suggests that the model was unable to accurately predict the actual network traffic values.



The results were slightly better for the UNSW-NB15 dataset. The model achieved an R^2 value of 0.3397, meaning it explained about 34% of the variation in the data. While this is an improvement compared to CICIDS2017, the scatterplot still shows a large amount of spread around the perfect prediction line. Many of the points appear in horizontal bands, which indicates that the model often predicted similar values even when the actual values were very different. The model also produced relatively large errors, with an RMSE of 131,794.79 and an MAE of 76,071.99, showing that its predictions were still not very accurate.

Overall, the Linear Regression model struggled to represent the complexity of real network traffic. The relationship between the actual and predicted values was weak, especially for the CICIDS2017 dataset. This suggests that network traffic patterns are influenced by more complicated relationships than a simple linear model can capture. Even though Linear Regression was useful as a starting point and provided a baseline for comparison, it was not a strong choice for modeling or detecting abnormal network activity. The results from both datasets show that more advanced machine learning models are needed to better capture the patterns and behaviors found in network traffic and cyberattacks.

K-Means Clustering and Hierarchical Clustering Results



The clustering models approached anomaly detection differently from the supervised learning models because they were not provided with labels indicating whether traffic was normal or malicious. Instead, they grouped records based on similarities in the network traffic features. To better visualize the clustering results, Principal Component Analysis (PCA) was used to reduce the data to two dimensions. The PCA cluster plot for the $k = 7$ K-Means model revealed several distinct groups of traffic records, although some overlap between clusters was still present. Most observations were concentrated within a few large clusters near the center of the visualization, while smaller groups of points extended farther away from the main clusters. These more isolated observations may represent unusual traffic patterns that could warrant additional investigation.

The hierarchical clustering results showed a similar overall structure. The dendrogram contained branches that remained separate until higher linkage distances, suggesting that certain groups of network traffic behaved differently from the majority of observations. Unlike Logistic Regression, Decision Trees, and Random Forests, clustering methods do not classify traffic as benign or malicious. Instead, they help identify relationships, patterns, and potential outliers within the data. This makes clustering useful for exploratory analysis and threat hunting, particularly in situations where labeled attack data is unavailable or when analysts are looking for previously unknown forms of suspicious activity.

Naive Bayes Results

The Naive Bayes models showed mixed performance depending on the dataset. On the UNSW-NB15 dataset, both the Gaussian and Multinomial versions were able to identify a large amount of attack traffic, but they approached the problem differently. Gaussian Naive Bayes tended to classify more traffic as attacks, which helped catch a larger number of malicious records but also increased the number of false positives. Multinomial Naive Bayes was more selective and correctly identified more normal traffic, although this came at the cost of missing additional attacks. The results highlight the tradeoff that often exists in intrusion detection systems between detecting as many threats as possible and avoiding excessive false alarms.

The CICIDS2017 dataset revealed a more significant weakness of both Naive Bayes models. The confusion matrices and prediction distributions showed that the models classified almost all traffic as malicious, regardless of whether it was actually benign or not. While this approach resulted in very few missed attacks, it also meant that legitimate traffic was rarely recognized correctly. This behavior suggests that the models were heavily influenced by the underlying class distribution and the simplifying assumptions made by Naive Bayes. Even though the overall results were not as strong as those produced by Random Forest, Decision Trees, or Logistic Regression, Naive Bayes still offers some practical value because it trains quickly and requires relatively little computational power. For that reason, it could be useful as an initial filtering step before traffic is analyzed by more accurate detection models.

Overall Results

Overall, the project showed that some machine learning techniques are much better suited for anomaly detection than others. The strongest results came from the supervised classification models, particularly Random Forest, which consistently achieved the highest accuracy and the most reliable attack detection rates on both datasets. Decision Trees also performed well and had the added benefit of being easy to interpret, making it possible to see exactly how a prediction was made. Logistic Regression was not as accurate as Random Forest, but it still performed well enough to serve as a useful baseline model and demonstrated that meaningful attack detection can be achieved even with a relatively simple approach.

The other models helped provide additional insight into the data, even though they were not as effective at directly identifying attacks. Linear Regression struggled to model network traffic accurately, which suggests that the relationships within the data are more complex than a simple linear equation can capture. K-Means and Hierarchical Clustering were useful for uncovering groups, outliers, and unusual patterns without relying on attack labels, making them valuable for exploratory analysis and threat hunting. Naive Bayes produced mixed results; it performed reasonably well on the UNSW-NB15 dataset but was heavily influenced by class imbalance in the CICIDS2017 dataset, causing it to favor attack predictions. Taken together, the results suggest that Random Forest would be the best choice for a production anomaly detection system, while clustering techniques could complement it by helping identify new or previously unseen threats that may not follow known attack patterns.

Thinking Questions Reflection

Now that the models have been tested and the results have been analyzed, it is possible to go back and answer the questions that guided this project. The results show how each model performed, what challenges they faced, and how successful they were at detecting malicious network activity.

Q1: Can this system detect cyber threats early enough to prevent real damage?

The results show that machine learning can help identify suspicious network activity at an early stage, giving security teams more time to respond before an incident becomes more serious. The best-performing models were able to catch most attacks in both datasets, which is encouraging from a cybersecurity standpoint. That said, none of the models were perfect. Some malicious traffic was still classified as normal, meaning certain threats could slip through undetected. For that reason, these models would likely work best as part of a larger security strategy where automated detection is combined with analyst review and investigation.

Q2: What is the acceptable balance between false positives and false negatives?

One thing that became clear during testing is that improving attack detection often comes with a tradeoff. Models that are more aggressive about flagging suspicious activity tend to catch more attacks, but they also generate more false alarms. On the other hand, models that are better at recognizing normal traffic are more likely to miss some malicious activity. In cybersecurity, missing a real attack is usually the bigger concern because an undetected threat can continue operating without anyone knowing it exists.

The Naive Bayes results on the UNSW-NB15 dataset showed this tradeoff well. Gaussian Naive Bayes identified a larger number of attacks, but it also flagged more legitimate traffic as suspicious. Multinomial Naive Bayes was more cautious and produced fewer false alarms, though it failed to detect some attacks that the Gaussian model caught. Which approach is preferred depends on the environment, but in many organizations it is easier to review extra alerts than to deal with the consequences of an attack that was never detected in the first place.

Q3: Which types of anomalies or attacks are easiest and hardest to detect?

The clustering results showed that some types of unusual activity are much easier to spot than others. In the anomaly visualization, the points that appeared far away from the larger groups were easy to identify because they behaved very differently from the majority of the network traffic. These observations stood out clearly and would likely be flagged quickly for additional review. When activity differs significantly from normal behavior, both clustering techniques and classification models tend to detect it more reliably.

The harder cases are the ones that do not look very different from normal traffic. Some malicious activity may blend into larger groups of legitimate network behavior, making it difficult to separate from everyday traffic patterns. This was reflected in the classification results, where certain attack records were incorrectly labeled as normal traffic. These findings suggest that obvious attacks are generally easier to detect, while low-volume, stealthy, or gradually evolving threats remain a challenge for both supervised and unsupervised machine learning approaches.

Q4: Can unsupervised learning detect previously unseen or evolving cyber threats?

One of the advantages of the clustering approach is that it does not need examples of known attacks to identify unusual behavior. Instead of learning what an attack looks like ahead of time, the model looks for observations that do not fit the patterns seen in the rest of the dataset. This makes it useful for situations where new threats emerge and labeled training data is not available.

The anomaly detection results provide a good example of this. Several observations appeared well outside the main clusters and were clearly different from the majority of the network traffic. While those points cannot automatically be labeled as attacks, they represent

activity that deserves closer examination. In a real-world environment, this type of information could help analysts focus on unusual behavior that might otherwise go unnoticed. The results suggest that unsupervised learning can play an important role in identifying new or evolving threats, especially when those threats do not match previously known attack patterns.

Q5: Which machine learning model provides the best performance for anomaly detection?

Looking at the results from both datasets, Random Forest was the model that performed the most consistently. It achieved the highest accuracy scores and was able to identify a large percentage of attack traffic without generating excessive classification errors. Compared to the other supervised models, it did a better job balancing missed attacks and false alarms, which is important when evaluating anomaly detection systems. The model also maintained strong performance on both CICIDS2017 and UNSW-NB15, suggesting that it was able to adapt well to different types of network traffic.

Decision Trees and Logistic Regression also produced solid results and were effective at detecting many attacks, but they were generally less consistent than Random Forest. The clustering methods provided useful insight into unusual traffic patterns and potential anomalies, but their purpose was different since they were not designed to directly classify traffic as normal or malicious. Based on the overall findings, Random Forest would be the strongest candidate for deployment because it offered the best combination of accuracy, reliability, and attack detection performance across the datasets used in this project.

Q6: How well do the models generalize to new, unseen data?

Using both the CICIDS2017 and UNSW-NB15 datasets made it possible to see whether the models could perform well outside of a single dataset. A model that only works well on one dataset may be learning dataset-specific patterns rather than behaviors that are common across different network environments. Comparing results across both datasets provided a better indication of how reliable each model might be when applied to new data.

Random Forest was the most consistent model throughout the project. It remained one of the top performers regardless of which dataset was used, suggesting that it was able to learn patterns that extended beyond a specific network environment. Logistic Regression also produced fairly stable results across both datasets, even though its overall accuracy was lower. The Naive Bayes models showed much larger differences in performance between datasets, particularly on CICIDS2017, where they struggled to distinguish between normal and malicious traffic. This suggests that some models are more sensitive to changes in data distribution than others, which can affect how well they perform when exposed to unfamiliar network traffic.

Q7: How does dimensionality reduction (PCA) impact anomaly detection performance?

PCA ended up making a noticeable difference in the clustering results. Before it was applied, the K-Means plots showed a large amount of overlap between clusters, making it difficult to see any clear patterns in the data. After reducing the dataset to a smaller number of principal components, the clusters became much easier to visualize and interpret. The separation between groups was more apparent, and unusual observations stood out more clearly than they did in the original feature space.

The PCA visualizations also helped reveal patterns that were not obvious when all of the original variables were considered at the same time. By reducing the complexity of the dataset, PCA removed some of the noise while preserving the information that contributed most to the variation in the data. This made the clustering results easier to analyze and improved the ability to identify potential anomalies. Based on these findings, dimensionality reduction appears to be a useful preprocessing step when working with large cybersecurity datasets that contain many features.

Q8: How can this system be integrated into a real-world cybersecurity workflow?

The results suggest that the models developed in this project could be incorporated into existing cybersecurity workflows rather than operating as standalone tools. Models such as Random Forest could be used to continuously monitor network traffic and identify activity that appears suspicious based on patterns learned during training. Events flagged by the model could then be prioritized for review, allowing security teams to focus their attention on the most potentially harmful activity instead of manually examining large volumes of network data.

The supervised models could also be used alongside clustering and anomaly detection techniques to provide a broader view of network behavior. While classification models are effective at recognizing known attack patterns, the clustering results showed that unusual observations can still be identified even when they do not resemble previously seen attacks. In practice, these outputs could be integrated into security dashboards, monitoring platforms, or alerting systems to support investigation and response efforts. Using both approaches together would provide a combination of attack classification and behavioral analysis, helping organizations detect known threats while also identifying activity that appears abnormal or unexpected.

Q9: How would cybersecurity analysts interpret and act on the model's results?

The visualizations created throughout this project demonstrate how machine learning results can be presented in a way that supports security analysis. Confusion matrices make it easier to see where models performed well and where classification errors occurred, while the clustering and anomaly detection visualizations highlight observations that differ from the majority of network traffic. Together, these outputs provide a clearer picture of network behavior and help identify areas that may require additional attention.

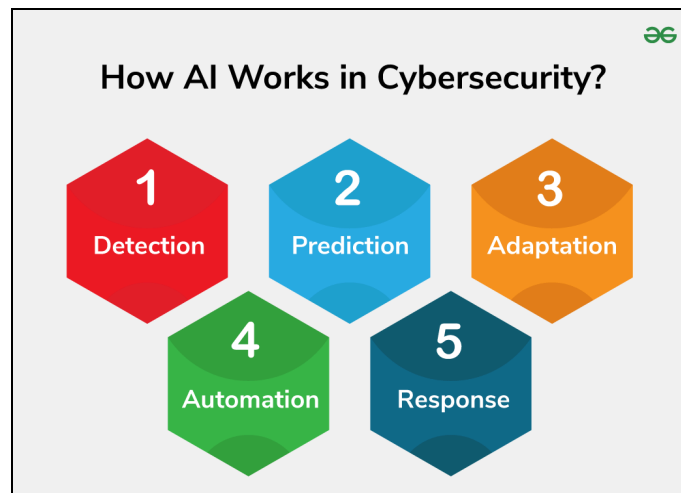
In a real-world environment, analysts do not rely on model predictions alone. Instead, the results would be used as one source of information alongside network logs, alerts, system activity, and other security data. A classification model might flag traffic as suspicious, but analysts would still need to review the surrounding context before determining whether a threat is present. Similarly, the anomaly detection visualizations could help direct investigations toward unusual activity that stands apart from normal traffic patterns. This approach allows analysts to focus their efforts more efficiently while still maintaining human oversight in the decision-making process.

Q10: What impact could this system have on improving cybersecurity in today's digital world?

The findings from this project show that machine learning can be a useful tool for helping organizations identify suspicious activity with large volumes of network traffic. As networks continue to expand and generate more data, it becomes increasingly difficult for security teams to manually review every event and determine which ones require attention. The models evaluated in this project were able to reduce that workload by highlighting traffic that appeared unusual or potentially malicious, allowing analysts to focus on the events that matter most.

Another important takeaway is not every machine learning approach was effective in every situation. The supervised models, particularly Random Forest, performed best when identifying known attack patterns because they were trained using labeled examples of malicious activity. On the other hand, the clustering and anomaly detection techniques were able to identify unusual behavior without relying on attack labels, making them useful for discovering new or evolving threats. The results from this project suggest that combining these approaches can provide broader coverage than relying on a single model alone. By using classification models to detect known threats and unsupervised methods to identify suspicious behavior that falls outside normal patterns, organizations can strengthen their ability to detect and respond to cyber threats in a constantly changing environment.

Conclusion



GeeksForGeeks AI in Cybersecurity Process

Machine learning is becoming a larger part of cybersecurity, and this project gave me the opportunity to see how different machine learning techniques can be used to analyze network traffic and detect suspicious activity. By using the CICIDS2017 and UNSW-NB15 datasets, I was able to compare several machine learning models and evaluate how well they identified normal and malicious traffic. The results showed that machine learning can be a useful tool for supporting cybersecurity efforts by helping distinguish potential attacks from legitimate network activity. Although no model performed perfectly, several of them were able to detect attacks with a high degree of accuracy, showing how machine learning can help security teams process large amounts of data more efficiently.

One of the biggest takeaways from this project was that some machine learning methods are much better suited for intrusion detection than others. Random Forest produced the strongest results on both datasets and was the most reliable model overall. The Decision Tree models also performed well and had the advantage of being easier to understand because their decision-making process could be visualized. Logistic Regression achieved reasonable results and provided a useful baseline for comparison. The clustering methods, K-Means and Hierarchical Clustering, approached the problem differently by looking for patterns and outliers without using attack labels. While these methods could not directly classify traffic as normal or malicious, they still helped reveal interesting structures within the data and showed how unusual traffic patterns might be identified.

This project also gave me experience with the full machine learning process, from preparing the data to analyzing the final results. A large amount of time was spent cleaning datasets, selecting features, training models, testing performance, and creating visualizations to explain the findings. Working through these steps helped improve my programming, problem-solving, and analytical skills. It also showed me how important data preparation is, since the quality of the input data can have a major impact on the results produced by a model.

Learning how to interpret the results and explain them clearly was just as important as building the models themselves.

Overall, this project was a valuable learning experience and helped me gain a better understanding of both machine learning and cybersecurity. It allowed me to apply concepts learned in my coursework to a real-world problem and see how these techniques can be used in practice. The experience also increased my confidence in working with data, building machine learning models, and evaluating their performance. As I continue preparing for a career in technology, cybersecurity, or data-related fields, the skills and knowledge gained from this project will be useful in future academic and professional work.

References

- Codecademy. (n.d.). *Log transformation in linear regression: When and how to use it*.
<https://www.codecademy.com/article/log-transformation-in-linear-regression-when-and-how-to-use-it>
- Data With Dylan. (2024). *How to merge datasets with Python Pandas [4 types]* [Video]. YouTube. <https://www.youtube.com/watch?v=xtbwWRXT5r0>
- DataCamp. (n.d.). *Random forests classifier in Python*.
<https://www.datacamp.com/tutorial/random-forests-classifier-python>
- GeeksforGeeks. (2023, December 26). *AI in cybersecurity*. GeeksforGeeks.
<https://www.geeksforgeeks.org/ethical-hacking/ai-in-cybersecurity/>
- GeeksforGeeks. (2026a). *Linear regression in machine learning*.
<https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/>
- GeeksforGeeks. (2026b). *Logistic regression in machine learning*.
<https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>
- Boros, G. (2023, July 25). *A simple introduction into supervised learning*. Medium.
<https://medium.com/@gerzson.boros/a-simple-introduction-into-supervised-learning-dcce83ee3ada>
- Niemi, J. (n.d.). *Simple linear regression using logarithms* [Video]. YouTube.
<https://www.youtube.com/watch?v=f7jzXnj4ga8>
- Pandas Development Team. (n.d.). *Merge, join, concatenate and compare. pandas Documentation*. https://pandas.pydata.org/docs/user_guide/merging.html
- Scikit-learn Developers. (n.d.). *Decision trees. Scikit-learn Documentation*.
<https://scikit-learn.org/stable/modules/tree.html>
- Scikit-learn Developers. (n.d.). *GaussianNB. Scikit-learn Documentation*.
https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html
- Scikit-learn Developers. (n.d.). *KNeighbors graph. Scikit-learn Documentation*.
https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.kneighbors_graph.html
- Scikit-learn Developers. (n.d.). *MultinomialNB. Scikit-learn Documentation*.
https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html
- Scikit-learn Developers. (n.d.). *RandomForestClassifier. Scikit-learn Documentation*.
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

Scikit-learn Developers. (n.d.). *Silhouette score*. *Scikit-learn Documentation*.
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html

Scikit-learn Developers. (n.d.). *SVC*. *Scikit-learn Documentation*.
<https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>

Galarnyk, M. J. (2019). *How to tune parameters in random forest, using Scikit-learn?* Stack Overflow.
<https://stackoverflow.com/questions/36107820/how-to-tune-parameters-in-random-forest-using-scikit-learn>

WebAsha Technologies. (n.d.). *Types of security logs: A complete guide for cybersecurity monitoring*.
<https://www.webasha.com/blog/types-of-security-logs-a-complete-guide-for-cybersecurity-monitoring>

W3Schools. (n.d.). *Python machine learning: Decision tree*.
https://www.w3schools.com/python/python_ml_decision_tree.asp